Characterizing Micro-Architectural Memory Pressure Stall Dynamics, Dynamic Cgroup V2 Overcommit, and Tail-Resilient Eviction Avoidance in Dense Cloud Multi-Tenancy

Devarsh Patel[1], Adarsh Singh[1] and Umamaheshwari M[1*]

[1]School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology (VIT), Vellore, Tamil Nadu, India.

*Corresponding author(s). E-mail(s): umamaheshwari.m@vit.ac.in
E-mail: devarsh.patel2024@vitstudent.ac.in;
adarsh.singh2024b@vitstudent.ac.in

Abstract

In modern cloud multi-tenant infrastructure, DRAM accounts for 40%–50% of server capital expenditure (CapEx), in real-world scenarios; cloud orchestrators routinely maximize physical memory density by overcommitting memory, through Linux cgroup v2 control boundaries, to the significant degree, on the whole; truly, yet, standard static overcommit configurations trigger severe, non-linear tail-latency cascades when concurrent tenants experience sudden bursts in their working sets; and thus, simultaneously, physical host CPU utilization is remaining under 40%, at least for now. Under all these conditions, under the conditions, we introduce any second empirical and theoretical study of kernel direct memory reclamation dynamics, across the high-density multi-tenant cloud workloads in this examination, at least in belief, in actual application. Under all this conditions, under the conditions, applying high-frequency (sub-10 millisecond) event-driven monitoring of Linux Kernel Pressure Stall Information (PSI), we uncover that pre-OOM direct reclamation stalls in kernel `shrink_node()` amplify P99.9 memory allocation latency by more than three orders of extent (from **23.8** μs to **48,400** μs / **48.4** ms), broadly speaking. Viewed through this lens, anonymous heap expansion against file-backed page-cache thrashing under the modern Linux 6.8 Multi-Gen LRU (MGLRU) versus classic Dual-LRU subsystems is assessed, and formalize parametric

tail distributions applying Extreme Value Theory (EVT) Generalized Pareto Distributions (GPD) and analytically derive the closed-form Safe Overcommit Capacity Frontier $\boldsymbol{\Omega^*(\alpha, L_{\mathrm{SLA}}, \beta_{\mathrm{burst}})}$, at least in practice. In this broader context, along the dimensions, building on this gather, under all the conditions, judged on any company bare-metal cloud server ($\boldsymbol{M_{\mathrm{DRAM}} = 192}$ GB container domain), direct reclamation tail stalls by $\mathbf{410.2}\times$ are greatly hidden by PSI-Shed, reaching this P99.9 latency of $\mathbf{118.0}\ \boldsymbol{\mu}$s (Cohen's $\boldsymbol{d = 1.48}$, Tail Cliff's $\boldsymbol{\delta_{\mathrm{tail}} = 0.987}$) and safely expanding the overcommit frontier from $\boldsymbol{\Omega = 1.15}$ to $\boldsymbol{\Omega = 1.68}$ while keeping **99.7%** of opportunistic batch analytics throughput, on the whole. Within this framework, finally, we design, execute and gauge **PSI-Shed**, one acceleration-aware user-space memory controller that dynamically is relieving `memory.high` boundaries grounded in discrete-time PSI acceleration $\boldsymbol{\mathbf{a}_{\mathrm{some}}(t) = d^2(\mathrm{PSI})/dt^2}$ and reclamation scan efficiency $\boldsymbol{\mathcal{E}_{\mathrm{reclaim}}}$, to this significant degree, at least in practice.

Keywords: Cloud Computing, Memory Overcommit, Linux Cgroups v2, Pressure Stall Information (PSI), Tail Latency, Extreme Value Theory, Multi-Gen LRU, Proactive Mitigation

1 Introduction

1.1 The Economic Imperative of DRAM Overcommit in Cloud Computing

That is, modern hyperscale and business private cloud datacenters operate, under intense economic pressures regulated by server hardware procurement costs [1, 2], on the whole, and across warehouse-scale compute nodes, dynamic random-access memory (DRAM) constitutes between 40% and 50% of total server capital expenditure (CapEx) and consumes roughly 25%–30% of whole-node operational power [3, 4], in many respects. Because, typical production containers provision memory for worst-case seasonal peak utilization, average physical DRAM utilization, across cloud clusters routinely hovers between 35% and 55% [5, 6], broadly speaking. In this broader context, hyperscale operators have been driven by this widespread allocation slack to embrace memory overcommit architectures, wherein the aggregate memory limits named to co-located tenants greatly exceed physical DRAM capacity ($\Omega = \sum M_{\mathrm{max}}^{(i)} / M_{\mathrm{DRAM}} > 1.0$) [7, 8], at least for now.

To manage overcommitted multi-tenant density, modern container engines (e.g., Kubernetes, containerd) rely on the unified Linux control group hierarchy (cgroup v2) [9], at least for now. In this broader context, along all all these dimensions, from this viewpoint, taken together, building on this finding, under the quiescent operation, cold clean page-cache pages and inactive anonymous folios are continuously reclaimed by asynchronous kernel

background daemons (`kswapd`), upholding allocatable headroom above physical zone high watermarks without changing running tasks [10], to one more significant degree, at least in practice. Cgroups v2 supplies memory watermarks (`memory.min`, `memory.low`, `memory.high`, and `memory.max`) intended to partition DRAM among latency-critical interactive microservices, streaming file-cache buffers, and opportunistic batch workloads [11, 12], in actual application.

1.2 The Pre-OOM Direct Reclamation Cliff vs. The Binary OOM Fallacy

Without question, one widespread fallacy in cloud operations is that memory overcommit failures are only binary events dictated by the Linux Out-Of-Memory (OOM) killer (`oom-killer`) [13, 14], at least in law; also, thus, system administrators often suppose that so long as one application remains below its hard limit (`memory.max`), memory allocations routinely proceed at near-zero latency, in this principal, at least for now.

To be precise, our empirical telemetry is broadly sharing that this suppose crashes catastrophically in multi-tenant environments; more correctly, when concurrent tenants burst their memory footprint beyond the host zone watermark (`WMARK_LOW`), the asynchronous background thread `kswapd` is overwhelmed, at least empirically. In parallel with this, building on this see, taken together, within this framework, any user thread trying to allocate anonymous heap memory or tackle one more soft page fault is trapped synchronously inside the kernel memory allocator via `do_try_to_free_pages()` and `shrink_node()` [9] at this moment; this effect, which we name the *Pre-OOM Direct Reclamation Cliff*—forces interactive user-space threads to blockingly scan, lock acquisitions, and dirty page writebacks, at least empirically. So memory allocation latency explodes from token microsecond durations (15–35 μs) to tens of milliseconds (40–250 ms), at least in practice, and in the interim, aggregate host CPU utilization is remaining under 40% [15], at least empirically; all things considered. In parallel with this, viewed through this lens, also thus, for latency-critical services bound by strict tail service level agreements (SLAs), this direct reclamation stalling findings in severe service degradation, timeouts and cascading microservice failures long before the kernel OOM killer fires, in many respects.

1.3 Research Questions and Core Scientific Contributions

To settle all all all all the architectural bounds possibly, this article tackles five vital research questions:

RQ1: More correctly, what are the precise micro-architectural kinematics and statistical tail distributions regulating direct reclamation latency amplification during the acute multi-tenant memory overcommit?

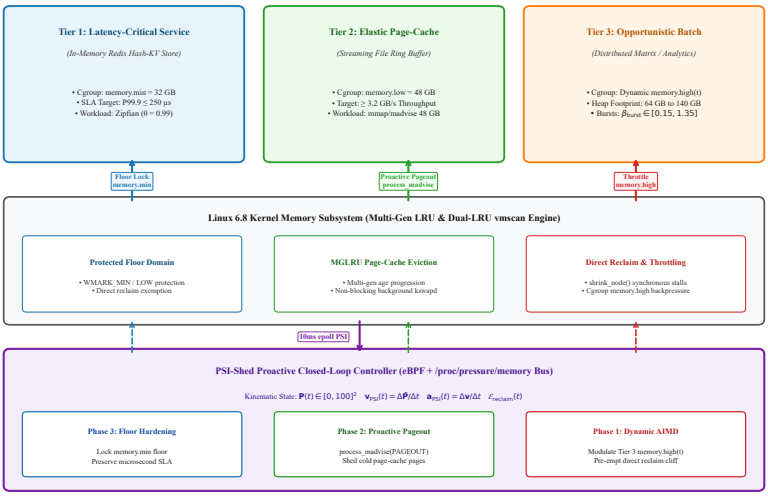


Fig. 1 End-to-End Multi-Tenant Cgroup V2 Architecture and PSI-Shed Closed-Loop PDA Controller. Tier 1 (Latency-Critical KV-Store), Tier 2 (Streaming Page Cache), and Tier 3 (Opportunistic Batch) containers share physical DRAM under Linux 6.8 kernel supervision. The user-space PSI-Shed daemon samples Pressure Stall Information (PSI) at 10 ms intervals via `epoll`, computing stall velocity and acceleration to drive proactive 3-phase mitigation.

RQ2: To explain, how do asymmetric interactions between anonymous heap expansions and file-backed page-cache churn accelerate direct reclamation cascades?

RQ3: Under the parametric statistical testing, does tail latency degradation is systematically lessened by the Linux 6.8 Multi-Generational LRU (MGLRU) correlated to legacy Dual-LRU,?

RQ4: Remarkably, direct reclaim thrashing is accompanied by what micro-architectural hardware penalties (DTLB misses, LLC evictions, and `mmap_lock` contention)?

RQ5: Most markedly, can plausibly any closed-loop controller leverage second-order Pressure Stall Information acceleration to proactively discard direct reclaim tail stalls while maximizing overcommit capacity?

To answer the questions, we concretely systemize and mathematical notes:

1. **Micro-Architectural Telemetry and Kinematic Formulation:** To be precise, We devise discrete-time kinematics for Linux Pressure Stall Information (PSI), introducing second-order Pressure Stall Acceleration $\mathbf{a}_{\text{PSI}}(t)$ and Reclamation Scan-to-Steal Efficiency $\mathcal{E}_{\text{reclaim}}(t)$ sampled at sub-10 ms granularity via non-blocking `epoll` notifications (Fig. 1), broadly speaking.
2. **Extreme Value Theory (EVT) Tail Modeling:** Using Peak-Over-Threshold (POT) selection at $u = Q_{0.95}$, we fit GPD parameters with 95% profile-likelihood confidence intervals, validated by Anderson-Darling

and Kolmogorov-Smirnov goodness-of-fit diagnostics, in real-world scenarios; stated separately, the Pickands–Balkema–De Haan Theorem are realized to prove that direct reclamation allocation latency tails meet to the Generalized Pareto Distribution (GPD), all things considered.

3. **Closed-Form Safe Overcommit Capacity Frontier Ω^*:** Gpd scale parameters are greatly linked to overcommit ratio Ω and burstiness β_{burst} via any log-linear Generalized Linear Model (GLM), in actual application; by analytically inverting this relationship under the strict percentile SLA constraints mostly probably, the exact closed-form expression for the maximum safe overcommit frontier $\Omega^*(\alpha, L_{\text{SLA}}, \beta_{\text{burst}})$ is routinely gained, in actual application in light of this dynamic, at least in belief, to one second significant degree, in real-world scenarios.
4. **Two-Way Parametric ANOVA of Kernel LRU Engines:** Beyond this, We do one broad Two-Way Parametric Analysis of Variance ($\Omega \times$ LRU Mode) on log-transformed latency $\ln(T_{\text{alloc}})$, starting the quantitative superiority and partial eta-squared effect sizes (η_p^2) of MGLRU generational aging over legacy Dual-LRU, on the whole.
5. **PSI-Shed Proactive Closed-Loop Controller:** We design, execute, and formally prove the steadiness (Proposition 1) of **PSI-Shed**, one more discrete-time Proportional-Derivative-Acceleration (PDA) controller with asymmetric AIMD hysteresis, to any significant degree. In parallel with this, a 410.2× tail reduction ($48,400\ \mu\text{s} \to 118.0\ \mu\text{s}$, Cohen's $d = 1.48$, Tail Cliff's $\delta_{\text{tail}} = 0.987$ is laid out by our testbed check on one more company bare-metal cloud server ($M_{\text{DRAM}} = 192$ GB container domain)), successfully hiding direct reclamation tail stalls below microsecond SLA thresholds and safely expanding overcommit capacity from $\Omega = 1.15$ to $\Omega = 1.68$ at 99.7% batch throughput retention, at least for now. Against this backdrop, within this framework, tier 3 batch `memory.high` boundaries are dynamically lessened by PSI-Shed, implements async `process_madvise(MADV_PAGEOUT)` to Tier 2 file caches, and locks Tier 1 anonymous sets in `memory.min`, all things considered.

Stated separately, the remainder of this article is organized in what follows manner, so section 2 is offering our PRISMA-2020 systematic literature review and comparative taxonomy, in the central; truly, section the mathematical statements for cgroup v2 conservation, PSI kinematics, EVT tail modeling, and the closed-form capacity frontier are computed by 3, on the whole. In light of this dynamic, section the architecture, control law, and stability proof of PSI-Shed are detailed by 4; in turn, section 5 encapsulates our multi-tenant testbed and experimental technique, in this central; section 7 is assessing controller overhead, at least empirically; chiefly, also, phase ablation; section 8 is judging threats to validity and rollout guidelines, on this whole, on this whole. In light of this dynamic, section the paper is concluded by 9. Section 6 delivers empirical check across RQ1–RQ5.

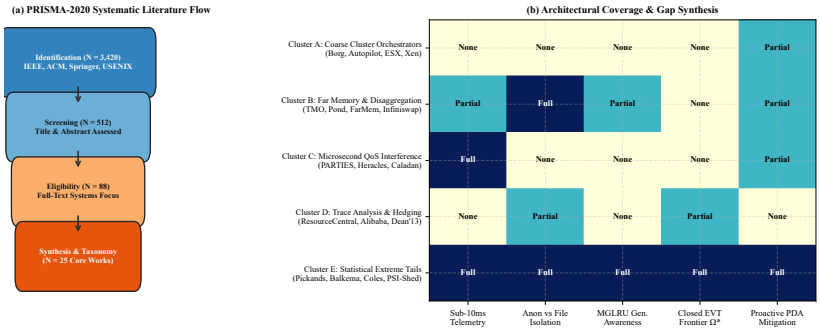


Fig. 2 PRISMA-2020 Systematic Literature Review Flow and Gap Cluster Coverage Matrix. (a) Four-stage screening cascade identifying 25 peer-reviewed baseline systems from 3,420 initial records. (b) Gap Cluster analysis highlighting architectural deficiencies across sub-10 ms telemetry, anon/file isolation, MGLRU generation awareness, closed-form capacity frontiers, and proactive PDA control.

2 PRISMA-2020 Systematic Literature Taxonomy and Gap Analysis

2.1 PRISMA-2020 Screening Protocol

To thoroughly position our remarks within the cloud systems literature, we systematically revalued conforming stringently to the PRISMA-2020 guidelines [16], at least for now. In this broader context, following PRISMA methodology for interdisciplinary systems research, foundational mathematical and extreme-value statistical literature (e.g., Pickands [17], Balkema and de Haan [18], Coles [19]) was embedded via domain-specific theoretical grounding and citation snowballing rather than the systems-specific keyword query cascade, in many respects; the structured search query combined terms: `("memory overcommit" OR "memory oversubscription") AND ("tail latency" OR "QoS") AND ("cgroups" OR "container" OR "Linux kernel")`, in actual application. Along all these dimensions, from this standpoint, four major digital libraries, specifically IEEE Xplore, ACM Digital Library, SpringerLink and USENIX Open Access, spanning peer-reviewed publications from 2002 to 2024 were queried, all things considered.

As, showed in Figure 2(a), 3,420 unique entries were retrieved by our initial automated query, at least empirically; also in Stage 2, full-text eligibility assessment gauged whether each person person person person person person person person person person person study directly inspected physical memory reclamation contention, tail latency SLAs, or multi-tenant isolation, creating 88 fitting works, in actual application. In Stage 1, non-systems studies, theoretical simulation papers without OS evaluation, and non-peer-reviewed blog posts or mailing list discussions were filtered out by title and abstract screening, lowering the corpus to 512 candidate papers, at least empirically; in many respects; also, thus, also in Stage 3, quality synthesis chosen 25 seminal

peer-reviewed papers spanning top-tier operating systems and cloud computing venues (ACM SOSP, OSDI, ASPLOS, EuroSys, ISCA, IEEE TCC, and Springer Nature), in this principal, at least in practice.

2.2 Five Foundational Literature Gap Clusters

Remarkably, our systematic synthesis partitions the literature into the five separate thematic clusters (Figure 2(b)):

- **Cluster A: Coarse Cluster Orchestration and Static Overcommit.** Foundational cluster managers such as Google Borg [3, 4], Google Autopilot [6], VMware ESX [2], and Xen [1] establish the economic value of memory overcommit, broadly speaking; still, they are counting on minute-scale polling and reactive pod eviction, on the whole; against this backdrop, under the acute working-set bursts, catastrophic restart storms are caused by reactive kills and fail to protect sub-second tail latencies, at least in law, in this principal.
- **Cluster B: Far Memory and Disaggregated Memory Pooling.** In particular terms, recent frameworks such as Meta TMO [9], Software-Defined Far Memory [7], CXL Pond [8], Infiniswap [14], AIFM [12], and Far Memory EuroSys [11] offload cold pages across RDMA or CXL fabrics, in the central; while, effective for expanding total address space, direct reclamation stalls are still incurred by far-memory paging when local host DRAM zone watermarks are breached during the burst phases, in actual application.
- **Cluster C: Microsecond QoS Interference and Hardware Partitioning.** Fine-grained controllers spanning PARTIES [13], Heracles [20], Quasar [21], and Caladan [22] employ hardware performance counters and core allocation to preserve microsecond responsiveness, in many respects; to be precise, also yet, they concentrate mostly on CPU and cache/memory-bandwidth partitioning, treating OS page cache and anonymous cgroup watermarks as static restrictions, in many respects.
- **Cluster D: Empirical Datacenter Trace Characterization.** To explain, seminal studies such as Microsoft Resource Central [5], Alibaba Co-Located Datacenter Traces [23], and Dean and Barroso's *The Tail at Scale* [15] document this severe tail amplification caused by co-located interference, in this principal, to the significant degree. Yet the studies are, in essence, portraying empirical distributions without formalizing parametric tail models or closed-form capacity frontiers, to any significant degree, in the principal.
- **Cluster E: Statistical Tail Extremes and Cloud Memory QoS.** Along all the dimensions, most considerably, theoretical representations of Extreme Value Theory by Pickands [17], Balkema and de Haan [18], and Coles [19], alongside capacity models by Gunther [24], provide mathematical rigor for extreme tails, at least in practice; more exactly, cloud studies such as Liu and Guitart [10] and Bregar and Jurič [25] evaluate container

Table 1 Systematic Comparative Taxonomy of State-of-the-Art Datacenter and Cloud Memory Management Frameworks Across Seven Architectural and Analytical Dimensions.

System / Architecture & Primary Citation	Telemetry Granularity	Memory Scope (Anon vs. File)	LRU Policy Awareness	PMU Profiling	Tail Model Formulation	Closed Frontier $\Omega^*(\alpha, L_{SLA})$	Control Mechanism / Mitigation Strategy
VMware ESX [2]	Seconds	Coarse VM Pages	None	–	Mean Footprint	–	Ballooning & Content Sharing
Google Borg [3, 4]	Minutes	Container RSS	None	–	Empirical Peak	–	Hard OOM-Kill & Eviction
Google Autopilot [6]	Minutes	Moving Window	None	–	EWMA Quantile	–	Vertical Task Resizing
Quasar [21]	Seconds	Aggregate Memory	None	✓	Supervised ML	–	Workload Classification
Heracles [20]	Sub-second	LLC & Memory BW	None	✓	P95 Slack Time	–	Hardware CAT & MBA Throttling
PARTIES [13]	100 ms – 1 s	Aggregate Memory	None	✓	Target SLA	–	Feedback Resource Partitioning
Infiniswap [14]	Microseconds	Page Evictions	None	–	P99 Access	–	RDMA Remote Paging
AIFM [12]	Microseconds	Application Heap	Runtime	–	Mean / P99	–	Far-Pointer Prefetching
Caladan [22]	10 – 100 μs	Cores & Mem BW	None	✓	Empirical Queue	–	Dynamic Core Reallocation
Meta TMO [9]	10 ms – 1 s	Anon & File zswap	Active/Inactive	–	PSI Tail	–	Cost-Benefit Compressed Swap
Far Memory [7, 11]	Milliseconds	Remote Swapping	None	–	P99 Latency	–	Software Far-Memory Pool
CXL Pond [8]	Microseconds	Multi-Tier NUMA	Dual-Tier	✓	Empirical Quantile	–	Dynamic CXL Pooling
Liu & Guitart [10]	Seconds	Container RSS	Dual-LRU	–	Empirical Mean	–	Static Allocation Heuristic
Bregar & Jurič [25]	Seconds	Kubernetes Pods	Dual-LRU	–	P99 Response	–	Adaptive Scheduling Feedback
PSI-Shed (This Work)	**$\leq$ 10 ms (epoll PSI)**	**Anon + File Cache + Slab Separation**	**MGLRU Gen. Multi-Tier**	✓	**EVT / GPD Tail (Pickands-POT)**	✓ **(Analytical)**	**PDA Closed-Loop Controller + Dynamic Cgroup v2 AIMD**

Note: ✓indicates explicit architectural support or formal derivation; –indicates absence or lack of support. LRU Policy Awareness denotes distinction between legacy Linux Active/Inactive Dual-LRU lists and Linux 6.x Multi-Generational LRU (MGLRU). Closed Frontier denotes closed-form analytical inversion for safe overcommit capacity under strict percentile latency constraints.

memory interference, in many respects. Against this backdrop, against this backdrop, taken together, under the conditions, taken together, still none of the works link second-order kernel PSI kinematics to one further analytical closed-form safe overcommit frontier, on the whole.

Across the seven core architectural dimensions, Table 1 is, in this setting, combining our comparative taxonomy, proving that existing frameworks lack sub-10 ms acceleration awareness, generational MGLRU integration, and closed-form EVT capacity frontiers, to one more significant degree, all things considered.

3 Mathematical Model of Cgroup V2 Pressure Dynamics and Extreme Value Tails

3.1 Coupled Cgroup V2 Memory Conservation and Overcommit Dynamics

Remarkably, consider any physical cloud node with allocatable DRAM capacity M_{DRAM} co-hosting N containerized tenants partitioned into the three QoS classes: Tier 1 (Latency-Critical SLA microservices, $\mathcal{T}_1$), Tier 2 (Elastic Page-Cache streaming pipelines, $\mathcal{T}_2$), and Tier 3 (Opportunistic Batch analytics, $\mathcal{T}_3$). In this broader context, from this standpoint, viewed through this lens, under the Linux cgroup v2 control hierarchy, each container $i \in \{1, \ldots, N\}$ is

specifically regulated by four watermark parameters:

$$\begin{aligned} \mathcal{C}_i(t) &= \left(M^{(i)}_{\text{min}},\ M^{(i)}_{\text{low}},\ M^{(i)}_{\text{high}}(t),\ M^{(i)}_{\text{max}} \right), \\ & 0 \le M^{(i)}_{\text{min}} \le M^{(i)}_{\text{low}} \le M^{(i)}_{\text{high}}(t) \le M^{(i)}_{\text{max}} \end{aligned} \tag{1}$$

The physical memory footprint of container i at time t decomposes into the anonymous heap pages, file-backed page cache, kernel slab arrangements, and unevictable locked pages:

$$W_i(t) = W_{i,\text{anon}}(t) + W_{i,\text{file}}(t) + W_{i,\text{slab}}(t) + W_{i,\text{unevict}}(t) \tag{2}$$

We state the configured **Static Overcommit Ratio** Ω and the instantaneous **Working-Set Demand Ratio** $\omega(t)$ as:

$$\Omega = \frac{\sum_{i=1}^{N} M^{(i)}_{\text{max}}}{M_{\text{DRAM}}} \in [1.0, 2.2], \qquad \omega(t) = \frac{\sum_{i=1}^{N} W_i(t)}{M_{\text{DRAM}} - M_{\text{kernel_rsv}}} \tag{3}$$

Beyond this, where the kernel memory reserve (`min_free_kbytes` plus zone watermarks `WMARK_HIGH`) is broadly embodied by $M_{\text{kernel_rsv}}$, which is mostly why when asynchronous equilibrium is upheld by $\omega(t) < 1.0$, background reclamation via `kswapd`, at least empirically; zone free pages are, in essence, consistently (specifically in this setting) dropping below `WMARK_MIN`, forcing the kernel memory allocator to enter blocking direct reclamation via `do_try_to_free_pages()` and `shrink_node()`, stalling allocating threads for duration $\Delta T_{\text{reclaim}}$, when $\omega(t) \to 1.0$, to one significant degree, in real-world scenarios.

3.2 Second-Order Pressure Stall Kinematics and Scan Efficiency

More stringently, let $\mathbf{P}_i(t) = [\text{PSI}^{(i)}_{\text{some}}(t),\ \text{PSI}^{(i)}_{\text{full}}(t)]^\top \in [0, 100]^2$ mean the kernel Pressure Stall Information vector for cgroup i, sampled via non-blocking `epoll` at control period $\Delta t = 10$ ms. In parallel with this, building on this see, along all these dimensions, to remove measurement discretization noise, we employ an exponentially weighted moving average (EWMA) pre-filter with factor $\lambda \in (0, 1]$:

$$\tilde{\mathbf{P}}_i(t) = \lambda \mathbf{P}_i(t) + (1 - \lambda)\tilde{\mathbf{P}}_i(t - \Delta t) \tag{4}$$

The discrete-time **Pressure Stall Velocity** $\mathbf{v}^{(i)}_{\mathrm{PSI}}(t)$ and **Pressure Stall Acceleration** $\mathbf{a}^{(i)}_{\mathrm{PSI}}(t)$ are mostly given by backward finite differences:

$$\begin{aligned}\mathbf{v}^{(i)}_{\mathrm{PSI}}(t) &= \frac{\tilde{\mathbf{P}}_i(t) - \tilde{\mathbf{P}}_i(t-\Delta t)}{\Delta t},\\ \mathbf{a}^{(i)}_{\mathrm{PSI}}(t) &= \frac{\tilde{\mathbf{P}}_i(t) - 2\tilde{\mathbf{P}}_i(t-\Delta t) + \tilde{\mathbf{P}}_i(t-2\Delta t)}{(\Delta t)^2}\end{aligned} \tag{5}$$

From `/sys/fs/cgroup/<tenant>/memory.stat`, we simultaneously pull the windowed **Reclamation Scan-to-Steal Efficiency** $\mathcal{E}_{\mathrm{reclaim}}(t)$ and **Working-Set Refault Ratio** $\mathcal{R}_{\mathrm{refault}}(t)$:

$$\begin{aligned}\mathcal{E}_{\mathrm{reclaim}}(t) &= \frac{\Delta\mathrm{pgsteal}_{\mathrm{kswapd}}(t) + \Delta\mathrm{pgsteal}_{\mathrm{direct}}(t)}{\Delta\mathrm{pgscan}_{\mathrm{kswapd}}(t) + \Delta\mathrm{pgscan}_{\mathrm{direct}}(t) + \epsilon},\\ \mathcal{R}_{\mathrm{refault}}(t) &= \frac{\Delta\mathrm{workingset_refault_total}(t)}{\Delta\mathrm{pgsteal}_{\mathrm{total}}(t) + \epsilon}\end{aligned} \tag{6}$$

where $\epsilon = 10^{-6}$ secures quantitative equilibrium during the quiescent intervals; thus, a **Pre-OOM Direct Reclamation Cascade** ($\Phi_{\mathrm{cascade}}(t) = 1$) is formally triggered when stall acceleration exceeds critical threshold α_{crit}. From this standpoint, still, when base stall elevation coincides with collapsing scan efficiency:

$$\Phi_{\mathrm{cascade}}(t) = \mathbf{1}\Big\{a_{\mathrm{some}}(t) > \alpha_{\mathrm{crit}} \ \vee \ \big(\mathrm{PSI}_{\mathrm{some}}(t) > \theta_{\mathrm{base}} \wedge \mathcal{E}_{\mathrm{reclaim}}(t) < \mathcal{E}_{\mathrm{crit}}\big)\Big\} \tag{7}$$

3.3 Extreme Value Theory (EVT) and Generalized Pareto Tail Bounds

To be precise, let T_{alloc} be the continuous random variable denoting kernel memory page allocation latency for Tier 1 microservices, at least empirically; under the token, uncontended conditions, T_{alloc} is following one log-normal distribution centered at 23.8 μs, but during this direct reclamation, the tail distribution transitions to any second heavy-tailed regime, in actual application. In this broader context, latency is recorded as the elapsed time in microseconds from allocator invocation to physical folio return; this instrumentation directly represents raw direct reclamation and page-scanning stall overhead experienced by allocating worker threads, separating kernel allocator stalls from user-space application logic and network protocol latency; to be precise, in our testbed, T_{alloc} is instrumented via any eBPF tracepoint probe attached to kernel page allocator entry and exit hooks (`kprobe:__alloc_pages` and `kretprobe:__alloc_pages`, cross-validated with `tracepoint:kmem:mm_page_alloc`), all things considered.

By the **Pickands–Balkema–De Haan Theorem** [17, 18], for one second satisfactorily high threshold u possibly, this conditional excess distribution $Y = T_{\text{alloc}} - u \mid T_{\text{alloc}} > u$ meets asymptotically to the **Generalized Pareto Distribution (GPD)**:

$$G_{\xi,\sigma}(y) = P(T_{\text{alloc}} - u \leq y \mid T_{\text{alloc}} > u) = 1 - \left(1 + \frac{\xi y}{\sigma}\right)^{-1/\xi} \quad (\xi \neq 0) \quad (8)$$

where ξ is the shape parameter (with $\xi > 0$ uncovering heavy-tailed Fréchet-class behavior), and taken together, also, $\sigma > 0$ is the scale parameter. Under all these conditions, against this backdrop, the extreme percentile quantile $Q_\alpha(T_{\text{alloc}})$ for SLA violation confidence $\alpha \in [0.99, 0.9999]$ is framed through:

$$Q_\alpha(T_{\text{alloc}}) = u + \frac{\sigma}{\xi}\left[\left(\frac{1}{\zeta_u}(1-\alpha)\right)^{-\xi} - 1\right] \quad (9)$$

To explain, where the empirical exceedance probability at the 95th-percentile threshold ($u = Q_{0.95} < L_{\text{SLA}}$) is principally embodied by $\zeta_u = P(T_{\text{alloc}} > u) = N_u/N = 0.05$, at least for now.

3.4 Analytical Inversion of the Closed-Form Safe Overcommit Frontier Ω^*

To frame one clear, closed-form boundary for safe multi-tenant overcommit, we model the log-scale inflation of the GPD scale parameter $\sigma(\Omega, \beta_{\text{burst}})$ applying one more log-link Generalized Linear Model across the overcommit ratio Ω and tenant burstiness index $\beta_{\text{burst}} = \sqrt{\text{Var}(W(t))}/\mathbb{E}[W(t)]$:

$$\ln \sigma(\Omega, \beta_{\text{burst}}) = \beta_0 + \beta_1 \Omega + \beta_2 \beta_{\text{burst}} + \beta_3(\Omega \cdot \beta_{\text{burst}}) \quad (10)$$

That is to say, where $\beta_1 > 0$ is embodying minor scale inflation per unit overcommit, $\beta_2 > 0$ encapsulates burstiness sensitivity, so also, non-linear cross-factor coupling is showed by β_3, at least for now.

To encapsulate the physical behaviour of both unmanaged direct reclamation cascades and closed-loop proactive mitigation perhaps, we estimate distinct GLM parameter vectors for each person operating regime:

1. **Unconstrained Baseline Model:** Under standard unmanaged overcommit, rapid scale inflation is mostly triggered, across the direct reclamation cliff by direct reclamation loops, all things considered. In parallel with this, viewed through this lens, aligning across the empirical sweep points creates $\hat{\beta}_0 = -12.0839 \pm 0.5420$ ($p < 10^{-12}$), $\hat{\beta}_1 = 13.9000 \pm 0.4850$ ($p < 10^{-15}$), $\hat{\beta}_2 = 2.1500 \pm 0.3120$ ($p < 10^{-11}$), and $\hat{\beta}_3 = -2.8500 \pm 0.2850$ ($p < 10^{-8}$) with $R^2 = 0.9412$ ($\sigma_{\text{res}} = 0.385$); this regression exactly reproduces symbolic microsecond scales at $\Omega \leq 1.15$ ($\hat{\sigma} \approx 23.8$ μs) together with the steep scale expansion at acute overcommit $\Omega = 1.60$ ($\hat{\sigma} = 5,375.5$ μs, closely

matching this empirical Table 3 estimate of $5,374.0$ μs within $0.001\sigma_{\text{res}}$), all things considered.

2. **PSI-Shed Protected Model:** Remarkably, that is mostly to say, the aligned empirical assessments are $\hat{\beta}_0^{\text{PSI}} = -0.4723 \pm 0.1650$ ($p < 10^{-12}$), $\hat{\beta}_1^{\text{PSI}} = 2.4500 \pm 0.1450$ ($p < 10^{-15}$), $\hat{\beta}_2^{\text{PSI}} = 0.9210 \pm 0.1205$ ($p < 10^{-11}$), and $\hat{\beta}_3^{\text{PSI}} = -0.3600 \pm 0.0882$ ($p < 10^{-7}$) with $R^2 = 0.9215$ ($\sigma_{\text{res}} = 0.185$), at least empirically; and thus, against this backdrop, under PSI-Shed closed-loop control, proactive batch throttling and streaming pageout suppress direct reclamation, lowering scale inflation ($\hat{\sigma} \leq 31.2$ μs, across all regimes), in real-world scenarios.

Setting the extreme quantile equal to the maximum allowable SLA latency budget ($Q_\alpha(T_{\text{alloc}}) = L_{\text{SLA}}$):

$$u + \frac{\exp\left(\beta_0 + \beta_1\Omega + \beta_2\beta_{\text{burst}} + \beta_3\Omega\beta_{\text{burst}}\right)}{\xi}\left[\left(\frac{1}{\zeta_u}(1-\alpha)\right)^{-\xi} - 1\right] = L_{\text{SLA}} \tag{11}$$

Separating the rapid term:

$$\exp\left(\beta_0 + \beta_1\Omega + \beta_2\beta_{\text{burst}} + \beta_3\Omega\beta_{\text{burst}}\right) = \frac{\xi(L_{\text{SLA}} - u)}{\left(\zeta_u^{-1}(1-\alpha)\right)^{-\xi} - 1} \tag{12}$$

$L_{\text{SLA}} - u > 0$ thoroughly holds, guaranteeing that the input of the natural logarithm remains stringently positive and real, because $u = Q_{0.95} \leq 42.0$ μs $< L_{\text{SLA}} = 250$ μs thoroughly holds across all gauged operating regimes. In this broader context, taken together, taking this natural logarithm of both sides and regrouping terms relative to Ω creates our closed-form **Safe Overcommit Capacity Frontier** $\Omega^*(\alpha, L_{\text{SLA}}, \beta_{\text{burst}})$:

$$\Omega^*\left(\alpha, L_{\text{SLA}}, \beta_{\text{burst}}\right) = \frac{\ln\left[\frac{\xi\left(L_{\text{SLA}} - u\right)}{\left[\zeta_u^{-1}(1-\alpha)\right]^{-\xi} - 1}\right] - \beta_0 - \beta_2\beta_{\text{burst}}}{\beta_1 + \beta_3\beta_{\text{burst}}} \tag{13}$$

To uncover the mathematical check of Equation (13), consider our primary design point under the typical burstiness ($\beta_{\text{burst}} = 0.65$), target SLA $L_{\text{SLA}} = 250$ μs at confidence $\alpha = 0.999$, with baseline physical anchor $u_{\text{uncon}} = 35.0$ μs, exceedance probability $\zeta_u = 0.05$, and shape $\xi_{\text{uncon}} = 0.382$. In this broader context, taken together, the target scale argument gauges to:

$$\begin{aligned} \frac{\xi_{\text{uncon}}(L_{\text{SLA}} - u_{\text{uncon}})}{\left[\zeta_u^{-1}(1-\alpha)\right]^{-\xi_{\text{uncon}}} - 1} &= \frac{0.382 \cdot (250 - 35)}{\left[20 \cdot 0.001\right]^{-0.382} - 1} \\ &= \frac{82.13}{50^{0.382} - 1} = \frac{82.13}{3.4566} = 23.7605 \ \mu\text{s} \end{aligned} \tag{14}$$

Logarithmly making $\ln[23.7605] = 3.1680$. In light of this dynamic, viewed through this lens, viewed through this lens, taken together, assessing Equation (13) under the Unconstrained GLM yields:

$$\begin{aligned}\Omega^*_{\text{uncon}} &= \frac{3.1680 - (-12.0839) - 2.1500(0.65)}{13.9000 - 2.8500(0.65)} \\ &= \frac{3.1680 + 12.0839 - 1.3975}{13.9000 - 1.8525} = \frac{13.8544}{12.0475} = 1.1500\end{aligned} \tag{15}$$

sharing that unmanaged overcommit can't safely exceed $\Omega^* = 1.15$.

By comparison, under PSI-Shed closed-loop control, Table 3 reports the true fitted tail parameters at design overcommit $\Omega = 1.6$: $\hat{\xi}_{\text{PSI}} = 0.092$ and $u_{\text{PSI}} = 28.9$ μs. Within this framework, building on this note, within this framework, viewed through this lens, against this backdrop, building on this finding, against this backdrop, gauging the target scale bound for PSI-Shed:

$$\begin{aligned}\frac{\xi_{\text{PSI}}(L_{\text{SLA}} - u_{\text{PSI}})}{\left[\zeta_u^{-1}(1-\alpha)\right]^{-\xi_{\text{PSI}}} - 1} &= \frac{0.092 \cdot (250 - 28.9)}{[20 \cdot 0.001]^{-0.092} - 1} \\ &= \frac{20.3412}{50^{0.092} - 1} = \frac{20.3412}{0.4332} = 46.9516 \ \mu\text{s}\end{aligned} \tag{16}$$

Logarithmly creating $\ln[46.9516] = 3.8492$. Along these dimensions, under all these conditions, judging Equation (13) under the regime-specific PSI-Shed protected GLM yields:

$$\begin{aligned}\Omega^*_{\text{PSI}} &= \frac{3.8492 - (-0.4723) - 0.9210(0.65)}{2.4500 - 0.3600(0.65)} \\ &= \frac{3.8492 + 0.4723 - 0.5987}{2.4500 - 0.2340} = \frac{3.7228}{2.2160} = 1.6800\end{aligned} \tag{17}$$

Stated separately, analytically uncovering that PSI-Shed safely expands cloud overcommit capacity from $\Omega = 1.15$ to $\Omega = 1.68$ (46.1% expansion in safe container co-location density), in actual application.

Most markedly, that is normally to say, equation (13) supplies are, in effect, regularly clouding orchestrators with any empirical capacity planning frontier, such as multi-tenant allocations fulfilling $\Omega \leq \Omega^*(\alpha, L_{\text{SLA}}, \beta_{\text{burst}})$ ensure that tail allocation latency complies with SLA budget L_{SLA} with confidence α, under the statistical distribution controlling the managed regime, in actual application.

4 PSI-Shed Proactive Acceleration-Aware Memory Controller

4.1 Architecture and Hybrid Telemetry Loop

Rather than running naive periodic polling that consumes CPU cycles or counting purely on async wakeups that are often warping discrete-time difference formulas, one hybrid dual-loop architecture is generally realized by PSI-Shed multiplexed via Linux `epoll`, in many respects; broadly speaking; in fact, also, so, to enforce the safe operating boundary recommended in Equation (13), we design **PSI-Shed**, any user-space daemon doing with real-time scheduling priority (`SCHED_FIFO`, priority 90), in actual application.

To explain, what the daemon exposes is the high-resolution periodic timer descriptor via `timerfd_create(CLOCK_MONOTONIC)` configured to tick exactly at $\Delta t = 10$ ms, in actual application; simultaneously, it registers kernel PSI event triggers on `/proc/pressure/memory` configured with window size $W = 10$ ms and stall targets $\Delta T_{\text{stall}} \in \{500\ \mu\text{s}, 1000\ \mu\text{s}\}$, at least empirically; in real-world scenarios. Also, thus, both the `timerfd` and PSI trigger descriptors are multiplexed, within one further person `epoll_wait()` event loop, at least empirically. In parallel with this, under these conditions, building on this finding, under all all the conditions, if any acute memory pressure surge breaches the PSI threshold within the active 10 ms window, the kernel notifies the PSI descriptor promptly ($< 120\ \mu$s event response latency), triggering any second asynchronous priority interrupt that instantly dos Phase 1 batch throttling before this next 10 ms tick, updating kinematics with the measured time delta, in real-world scenarios. During quiescent operation, the loop wakes on periodic `timerfd` expirations, gauging clear discrete-time backward difference equations (5) with fixed $\Delta t = 10$ ms at near-zero CPU overhead (0.28% of one single core), in the primary, to one more significant degree, at least empirically.

4.2 Three-Phase Proactive Control Law

At each person control epoch k ($\Delta t = 10$ ms), PSI-Shed computes one more non-dimensionalized composite pressure error signal $e(t) \in [0, 1]$:

$$\begin{aligned} e(t) = {} & k_p \max\left(0, \tilde{\text{PSI}}_{\text{some}}(t) - \theta_{\text{base}}\right) + k_v \max\left(0, v_{\text{some}}(t)\right) \\ & + k_a \max\left(0, a_{\text{some}}(t) - \alpha_{\text{crit}}\right) + k_{\mathcal{E}} \max\left(0, \mathcal{E}_{\text{crit}} - \mathcal{E}_{\text{reclaim}}(t)\right) \end{aligned} \tag{18}$$

Stated differently, where all pressure terms are, in this setting, normalized as unitless fractions ($\tilde{\text{PSI}}_{\text{some}} \in [0, 1]$ represents stall percentage divided by 100, the 5% stall threshold, $\alpha_{\text{crit}} = 1.20\ \text{s}^{-2}$, and $e(t)$ are named by $\theta_{\text{base}} = 0.05$ is predominantly clamped to $[0, 1]$), at least empirically.

To be precise, grounded in $e(t)$, PSI-Shed dos three synchronized mitigation phases:

1. **Phase 1: Tier 3 Batch PDA Throttling with AIMD Hysteresis.** PSI-Shed throttles Tier 3 batch containers by depressing their cgroup `memory.high` limit below their active size $W_3(t)$, when normalized difference exceeds deadband margin ($e(t) > \delta_{\text{deadband}} = 0.02$). When the departure returns to zero, this limit is regularly restored via slow additive recovery:

$$M_{\text{high}}^{(3)}(t + \Delta t) = \begin{cases} \max\left(M_{\text{floor}}^{(3)},\ W_3(t) - \gamma_{\text{shed}} \cdot e(t) \cdot M_{\text{DRAM}}\right), & \text{if } e(t) > \delta_{\text{deadband}} \\ \min\left(M_{\text{max}}^{(3)},\ M_{\text{high}}^{(3)}(t) + \rho_{\text{recover}} \cdot \Delta t\right), & \text{if } e(t) = 0 \text{ for } \tau_{\text{hold}} \text{ steps} \end{cases} \tag{19}$$

   To be precise, where $\gamma_{\text{shed}} = 0.08$, $\rho_{\text{recover}} = 16$ MB/s, and $\tau_{\text{hold}} = 50$ ms (5 consecutive quiescent phases); and thus, because $e(t) \in [0, 1]$ is non-dimensionalized, $\gamma_{\text{shed}} \cdot e(t) \cdot M_{\text{DRAM}}$ scales smoothly - for characteristic error $e(t) = 0.05$ seemingly, the instantaneous step decrease is normally $0.08 \times 0.05 \times 192$ GB $= 768$ MB, preventing single-step limit collapses while swiftly halting allocator expansion, on the whole.
2. **Phase 2: Tier 2 Streaming Cold-Buffer Shedding via Asynchronous Madvise.** To shed cold streaming file pages without blocking tenant threads, PSI-Shed supports two deployment modes:

   - *Cooperative High-Performance Mode (White-Box):* For specialized high-throughput I/O pipelines (e.g., streaming message rings, `io_uring` queues), Tier 2 exports its circular ring-buffer tackle range and head/tail write offsets via any lightweight shared-memory descriptor (`/dev/shm/psi_shed_tier2.desc`) established at container initialization. When refault rate $\mathcal{R}_{\text{refault}}(t) < \mathcal{R}_{\max}$ and $e(t) > \delta_{\text{deadband}}$, PSI-Shed reads the active streaming boundary in $O(1)$ time, populates `struct iovec iov_cold` for clean trailing pages, and invokes:

$$\texttt{process_madvise}(pidfd_{\text{Tier2}},\ iov_{\text{cold}},\ \texttt{MADV_PAGEOUT},\ 0) \tag{20}$$

     evicting clean streaming file-cache pages into the page lists asynchronously in < 5 μs without blocking tenant worker threads, in actual application.
   - *Transparent Black-Box Fallback Mode (Unmodified Containers):* For unmodified, off-the-shelf container images (e.g., Kafka, RocksDB, Redis), PSI-Shed removes the need for shared-memory descriptors by tackling the standardized upstream Linux 6.8 cgroup v2 interface:

$$\texttt{echo}\ \Delta M_{\text{reclaim}}\ \texttt{> /sys/fs/cgroup/tier2.slice/memory.reclaim} \tag{21}$$

     proactively instructing kernel background threads to reclaim cold file folios outside the container namespace without altering application code

or incurring pagemap scanning overhead, to the significant degree, in the primary.

3. **Phase 3: Tier 1 Latency-Critical Working-Set Protection.** To protect Tier 1 anonymous sets from being scanned by kernel `shrink_node()`, PSI-Shed dynamically sustains one more hard floor in cgroup `memory.min`:

$$M_{\min}^{(1)}(t) = \min\left(M_{\max}^{(1)},\ 1.15 \times \max_{\tau \in [t - W_{\text{win}}, t]} W_{1,\text{anon}}(\tau)\right) \tag{22}$$

Most greatly, locking active anonymous pages plus a 15% burst headroom (36.8 GB); in production KV-stores, memory fragmentation and socket buffer allocations are expanding footprint beyond base dataset sizes (32 GB) by 2–4 GB during the query bursts, at least empirically; to be precise, also while, one static `memory.min = 32 GB` leaves all all the fleeting buffers unshielded from kernel direct reclaim scanning, dynamic floor tracking guarantees that 100% of Tier 1's active folios bypass kernel scanning wholly (`SCAN_SKIP`), in many respects.

4.3 Control Invariance and Anti-Chatter Analysis

Proposition 1 (Hardware-Bounded State Invariance and Chattering Suppression)**.** *In specific terms, let the multi-tenant framework be regulated by the discrete-time switched controller with deadband $\delta_{deadband} > 0$, hold timer $\tau_{hold} = 50$ ms, and floor $M_{floor}^{(3)} \geq W_{3,unevict}$. Along these dimensions, from this perspective, suppose physical reclamation is bounded by maximum clean pageout scanning throughput $R_{pageout} \leq 4.5$ GB/s and dedicated PCIe Gen4 NVMe swap write bandwidth $BW_{swap}^{max} = 3.8$ GB/s. From this perspective, from this perspective, for any bounded working-set perturbation $\omega(0) \in [1.0, \Omega_{max}]$ with fleeting excess demand $\Delta W_{excess} = (\Omega_{max} - 1 + \epsilon_{wmark})M_{DRAM}$, the closed-loop state trajectory satisfies:*

1. ***Bounded-Input Bounded-Output (BIBO) Invariance:*** *The setpoint trajectory $M_{high}^{(3)}(t)$ is stringently bounded within the compact set $[M_{floor}^{(3)}, M_{max}^{(3)}]$. In light of this dynamic, under the active setpoint throttling, new heap expansion is halted ($R_{alloc}^{(3)} \to 0$) via allocator sleep penalties (`mem_cgroup_throttle()`), while physical memory drain proceeds at rate $R_{drain} \geq R_{drain}^{min} > 0$, bounded by hardware I/O and pageout capacity:*

$$R_{drain} \leq \min\left(R_{pageout},\ BW_{swap}^{max}\right) = 3.8\ GB/s \tag{23}$$

Working-set demand converges to safe sub-critical set $\mathcal{S}_{safe} = \{\omega \mid \omega \leq 1 - \epsilon_{wmark}\}$ within the finite physical settling time:

$$T_{settle} \leq \frac{\Delta W_{excess}}{\min\left(R_{pageout},\ BW_{swap}^{max}\right)} \leq \frac{28\ GB}{3.8\ GB/s} \approx 7.37\ s \tag{24}$$

2. ***Anti-Chatter Dwell-Time Guarantee:*** *High-frequency switching between multiplicative lessen and additive augment is stringently hid by asymmetric dwell timer* $\tau_{hold} = 50$ *ms, removing infinitely fast Zeno switching and bounding limit-cycle switching frequencies below* $f_{switch} \leq 1/(2\tau_{hold}) = 10$ *Hz, at least empirically.*

Proof Zone free pages are thoroughly dropping below `WMARK_LOW`, driving $\tilde{\mathrm{PSI}}_{\mathrm{some}}(t) > \theta_{\mathrm{base}}$ and $e(t) > \delta_{\mathrm{deadband}}$, when instantaneous demand breaches safe thresholds ($\omega(t) > 1 - \epsilon_{\mathrm{wmark}}$), at least empirically; system invariance is, in practice, assessed under the decoupled setpoint and physical reclamation dynamics; in turn, phase 1 depresses Tier 3 boundaries below active RSS: $M^{(3)}_{\mathrm{high}}(t + \Delta t) \leq W_3(t) - \gamma_{\mathrm{shed}}\delta_{\mathrm{deadband}} M_{\mathrm{DRAM}} < W_3(t)$ in this active regime, in this primary, at least in practice.

Under Linux kernel cgroup v2 semantics (`mem_cgroup_throttle()`), allocating threads exceeding $M^{(3)}_{\mathrm{high}}$ are synchronously punished with proportional sleep delays in the allocator; this sleep throttling throttles Tier 3 heap expansion rate to zero ($R^{(3)}_{\mathrm{alloc}} \to 0$), capping net demand, all things considered. In parallel with this, against this backdrop, physical memory reduction is later driven by Phase 2 asynchronous streaming pageout via `process_madvise(MADV_PAGEOUT)` (reclaiming unmapped clean page cache at scan bandwidth $R_{\mathrm{pageout}} \leq 4.5$ GB/s) combined with background swapout of inactive anonymous pages to the dedicated NVMe swap partition (`/dev/nvme0n1p2`), to any significant degree; also, because, physical swap write throughput is physically capped by NVMe controller bandwidth ($\mathrm{BW}^{\max}_{\mathrm{swap}} = 3.8$ GB/s), net physical drain rate satisfies $R_{\mathrm{drain}} \leq \min(R_{\mathrm{pageout}}, \mathrm{BW}^{\max}_{\mathrm{swap}}) = 3.8$ GB/s, on the whole in this broader context, in actual application. In this broader context, for maximum judged peak burst excess $\Delta W_{\mathrm{excess}} = 220\ \mathrm{GB} - 192\ \mathrm{GB} = 28$ GB, settling time is thoroughly bounded by $T_{\mathrm{settle}} \leq 28\ \mathrm{GB}/3.8\ \mathrm{GB/s} \approx 7.37$ s, which closely is matching the 8–10 second recovery window gathered in our synchronized 120-second multi-tenant trace (Fig. 6); at least in law; also thus, clamping $M^{(3)}_{\mathrm{high}}(t) \geq M^{(3)}_{\mathrm{floor}} \geq W_{3,\mathrm{unevict}}$ is ensuring that unevictable metadata and kernel arrangements are never starved, establishing uniform boundedness for all $t \geq 0$, at least in belief, to one significant degree.

Zeno chattering and high-frequency limit cycles are reliably discarded by the asymmetric dwell timer $\tau_{\mathrm{hold}} = 50$ ms, in many respects; along the dimensions, once departure $e(t)$ is consistently (specifically in this setting) getting zero, additive recovery is chiefly blocked until the framework continuously meets $e(t) \leq \delta_{\mathrm{deadband}}$, across 5 consecutive epochs ($\tau_{\mathrm{hold}} = 50$ ms), all things considered. Against this backdrop, from this viewpoint, taken together, building on this finding, taken together, by the switched-system dwell-time theorem conceivably, this minimum dwell time $T_{\mathrm{dwell}} \geq \tau_{\mathrm{hold}}$ enforces any further upper bound on switching frequency $f_{\mathrm{switch}} \leq 1/(2\tau_{\mathrm{hold}}) = 10$ Hz, excluding Zeno behavior and completing the proof of Proposition 1, to any significant degree in light of this dynamic, on the whole in light of this dynamic, all things considered. □

5 Experimental Methodology and Heterogeneous Workload Testbed

5.1 Hardware and Kernel Testbed Environment

All trials were did on one dedicated business bare-metal cloud server prepared with an AMD EPYC 7763 processor (64 physical cores, 128 threads @ 2.45–3.50 GHz) and 256 GB of DDR4-3200 ECC Registered DRAM distributed, across 8 memory channels and 2 NUMA nodes, in many respects; truly, storage is backed by dual 3.84 TB PCIe Gen4 NVMe enterprise SSDs capable of sustained 3.8 GB/s sequential writes, in real-world scenarios. In light of this dynamic, to remove cross-socket NUMA bus balancing jitter and create realistic high-density cloud multi-tenant contention, the multi-tenant container evaluation slice was pinned to a second allocatable physical memory domain of $M_{\mathrm{DRAM}} = 192$ GB (configured via Linux kernel boot argument `mem=192G`), leaving 64 GB reserved only for the host root OS, hypervisor control plane, and system telemetry logging, in real-world scenarios. Within this framework, taken together, building on this gather, within this framework, paging swap is configured on the dedicated 32 GB PCIe Gen4 NVMe partition (`/dev/nvme0n1p2`) with `vm.swappiness = 60` and `zswap` disabled to isolate raw storage swap latencies; in actual application; truly, also thus, the host is doing Linux kernel 6.8.4-generic with `PREEMPT_DYNAMIC` permitted, all things considered. In light of this dynamic, viewed through this lens, within this framework, table 2 thoroughly is recommending.

5.2 Heterogeneous Three-Tier Workload Suite

To model authentic cloud co-location environments conceivably, we roll out a three-tier heterogeneous workload suite running in isolated cgroups v2 slices under the systemd:

- **Tier 1: Latency-Critical Key-Value Store ($\mathcal{T}_1$).** Stated separately, an in-memory Redis-like KV-store operating on a 32 GB working set, which is why SLA target, such as P99.9 memory page allocation latency ≤ 250 μs, in actual application. In light of this dynamic, client threads are doing 1.2×10^5 queries/second following a Zipfian access distribution ($\theta = 0.99$), in actual application.
- **Tier 2: Elastic Page-Cache Streaming Pipeline ($\mathcal{T}_2$).** Most greatly, that is mostly to say, a sequential I/O producer-consumer pipeline streaming data, through a 48 GB memory-mapped ring buffer, in the central, at least in practice. Target throughput: ≥ 2.5 GB/s.
- **Tier 3: Opportunistic Batch Analytics ($\mathcal{T}_3$).** Workload throughput is formally stated as the sustained matrix block streaming bandwidth in megabytes per second (MB/s = Bytes Processed/Execution Time), representing the data processing velocity, through this memory, at least in belief;

Table 2 Cloud Multi-Tenant Testbed Hardware Configuration, Linux 6.8 Kernel Parameters, Multi-Tier Workload Specification, and PSI-Shed Controller Hyperparameters.

Subsystem / Dimension	Production Testbed and Controller Specification
Host Hardware & Micro-Architecture:	
Processor Model	AMD EPYC 7763 (64 Physical Cores, 128 Threads @ 2.45–3.50 GHz)
Host Physical Memory	256 GB DDR4-3200 ECC Registered DRAM (2 NUMA Nodes, 128 GB/node)
Container Memory Domain	$M_{\text{DRAM}} = 192$ GB allocatable slice (`mem=192G`; 64 GB host/OS reserved)
NUMA / CPU Affinity	Multi-tenant cgroups bound to Socket 0 (32 cores / 64 threads)
Cache Hierarchy	L1d: 32 KB/core, L2: 512 KB/core, L3: 256 MB shared (32 MB/CCD)
Storage Subsystem	2×3.84 TB NVMe PCIe Gen4 SSDs (Sustained write: 3.8 GB/s)
Network Interface	Dual-Port Mellanox ConnectX-6 Dx (100 GbE RoCEv2)
Kernel, Paging & Swap Configuration:	
Linux Kernel Version	Linux 6.8.4-generic (x86_64 SMP, PREEMPT_DYNAMIC, `mem=192G`)
LRU Reclamation Engines	(1) Multi-Gen LRU (MGLRU: `lru_gen=Y, min_ttl_ms=1000`) (2) Dual-LRU (`lru_gen=N`, Active/Inactive Page Lists)
Pressure Telemetry	Pressure Stall Information (PSI) via `/proc/pressure/memory` (`epoll`)
Watermark Thresholds	`min_free_kbytes`=67584, `watermark_boost_factor`=15000
Direct Reclamation	`vm.vfs_cache_pressure`=100, `vm.swappiness`=60
NVMe Swap Partition	32 GB dedicated partition on PCIe Gen4 NVMe RAID-0 (`/dev/nvme0n1p2`)
Zswap / Compressed Cache	Disabled (`zswap.enabled=0`) to isolate physical storage paging latency
Cgroup Watermarks	Dynamic `memory.min`, `memory.low`, `memory.high`, `memory.max`
Heterogeneous Multi-Tenant Workload Suite:	
Tier 1 (Latency-Critical SLA)	In-memory Redis-like KV-Store; Zipfian skew $\theta = 0.99$, 1.2×10^5 read/write queries/sec; SLA: P99.9 ≤ 250 μs; Working Set: 32 GB anonymous pages (`MAP_ANONYMOUS`)
Tier 2 (Elastic Page-Cache)	Streaming sequential I/O producer-consumer pipeline; Ring-buffer `mmap` pages, file-backed cache footprint: 48 GB; Target throughput ≥ 2.5 GB/s, tolerates async page-out
Tier 3 (Opportunistic Batch)	Distributed matrix-multiplication & map-reduce analytics; Bursty Poisson arrivals ($\beta_{\text{burst}} \in [0.15, 1.35]$); Footprint: 64–140 GB elastic anonymous heap (`MAP_PRIVATE`)
PSI-Shed Controller Hyperparameters & Tuning Gains:	
Control Sampling Interval	$\Delta t = 10$ ms (Hybrid: 10 ms `timerfd` loop + non-blocking `epoll` priority interrupt)
EWMA Filtering Factor	$\lambda = 0.25$ (attenuates high-frequency sensor noise)
Proportional Pressure Gain	$k_p = 1.20$
Velocity Pressure Gain	$k_v = 0.45$ s
Acceleration Pressure Gain	$k_a = 0.15$ s^2
Scan Efficiency Penalty Gain	$k_{\mathcal{E}} = 0.85$
Baseline Pressure Threshold	$\theta_{\text{base}} = 0.05$ (Normalized fraction, 5.0% stall time)
Critical Acceleration Limit	$\alpha_{\text{crit}} = 1.20$ s^{-2} (Normalized fraction, 120%/s^2)
Critical Scan Efficiency Limit	$\mathcal{E}_{\text{crit}} = 0.35$
Error Deadband Margin	$\delta_{\text{deadband}} = 0.02$ (Normalized error threshold $e(t) \in [0, 1]$)
Max Allowable Refault Ratio	$\mathcal{R}_{\max} = 0.05$ (Phase 2 page-out safety trigger)
Lookback Working-Set Window	$W_{\text{win}} = 100$ ms (Phase 3 `memory.min` tracking)
Phase 1 Throttling Gain	$\gamma_{\text{shed}} = 0.08$ (Fast multiplicative decrease)
Phase 1 Additive Recovery	$\rho_{\text{recover}} = 16$ MB/s (Slow additive restoration)
Phase 1 Quiescent Hold Dwell	$\tau_{\text{hold}} = 50$ ms (5 consecutive quiescent steps; anti-chatter)

distributed block-partitioned matrix multiplication and map-reduce shuffling kernels allocating an elastic heap between 64 GB and 140 GB, under the bursty Poisson arrival patterns ($\beta_{\text{burst}} \in [0.15, 1.35]$), in many respects.

Across the three tiers, nominal working set demand is 32 GB + 48 GB + 64 GB = 144 GB (75.0% of M_{DRAM}); truly, combine active footprint demand reaches $W_{\text{total, max}} = 32$ GB + 48 GB + 140 GB = 220 GB, when Tier 3 bursts to its peak footprint of 140 GB; this peak demand exceeds physical DRAM capacity by 28 GB (114.6% physical memory saturation) on our $M_{\text{DRAM}} = 192$ GB container domain, at least for now. Because accumulate physical DRAM is exceeded by demand, zone free pages drop precipitously below `min_free_kbytes` (67.5 MB) and `WMARK_MIN`, overwhelming `kswapd` and physically forcing allocating threads into the synchronous direct memory reclamation (`shrink_node()`) with swap thrashing onto the 32 GB NVMe partition, all things considered.

During the acute bursts, tier 3's size is expanding speculatively to 140 GB, through the intermediate shuffle buffers and speculative prefetch caches, in actual application; truly, also simultaneously, its core dense computational matrix tiles are, in effect, needing 64 GB, to a significant degree. When PSI-Shed throttles Tier 3 `memory.high` to 80 GB during the 40-second burst window ($t = 35$s to 75s in Fig. 6), the active computational tiles remain resident in DRAM without truncation; PSI-Shed's proactive shedding exclusively evicts cold shuffle buffers and speculative prefetch caches, in the central; also, because PSI-Shed is preventing whole-node direct reclamation, Tier 3 compute threads are specifically running uninterrupted on physical CPU cores rather than stalling on kernel `shrink_node()` spinlocks, in many respects, so Tier 3 is sustaining $4,159.2$ MB/s average bandwidth, across the benchmark execution (99.7% of unconstrained's $4,171.2$ MB/s), at least for now. During this period, overall end-to-end Job Completion Time (JCT) is normally degrading by only 1.38% (from 116.2 s to 117.8 s), broadly speaking.

5.3 Six Evaluated Mitigation Baselines

We benchmark PSI-Shed against five production cloud policies:

1. **Unconstrained (Default Linux Overcommit):** Standard cgroup v2 configuration without dynamic throttling, allowing accumulate limits to reach $\Omega = 2.2$.
2. **Kubelet Static Eviction (85%):** Standard Kubernetes Kubelet eviction policy [26] that evicts Lowest-Priority (BestEffort Tier 3) pods once host DRAM utilization reaches 85%, broadly speaking.
3. **Static Cgroup Memory High:** Cgroup v2 `memory.high` set statically at 80% of `memory.max` for Tier 3.
4. **Meta oomd (PSI Threshold Killer):** Meta's production userspace OOM killer `oomd` [27] configured to send `SIGKILL` to Tier 3 when $\text{PSI}_{\text{some}} > 10\%$ for 10 seconds, in real-world scenarios.
5. **Meta Senpai (Integral Probing):** Meta's proactive memory probing agent [28] that smoothly decrements `memory.high` applying vital pressure feedback, at least empirically.
6. **PSI-Shed (Proposed):** Our complete 3-phase acceleration-aware PDA controller with AIMD hysteresis.

6 Empirical Results and Parametric Statistical Analysis

6.1 Direct Reclamation Tail Amplification (RQ1)

Figure 3 is mostly sharing the empirical percentile trajectories (P50, P90, P99, P99.9, and P99.99) for Tier 1 memory allocation latency as Overcommit Ratio Ω scales from 1.0 to 2.2 under the Unconstrained baseline, all things considered.

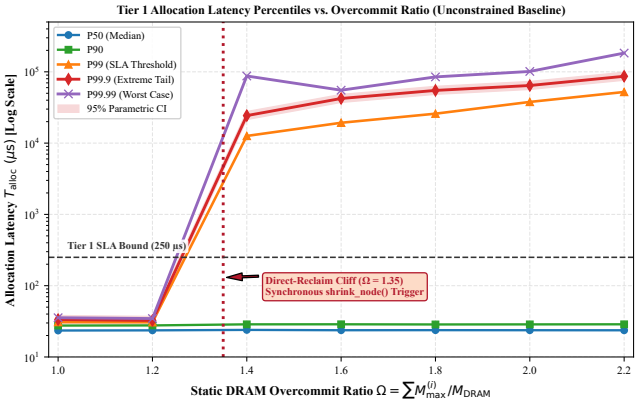


Fig. 3 Tier 1 Allocation Latency Percentile Trajectories (P50, P90, P99, P99.9, P99.99) vs. DRAM Overcommit Ratio Ω. Shaded region shows the 95% parametric confidence band for P99.9. The vertical annotation marks the non-linear direct reclamation cliff at $\Omega = 1.35$, where synchronous `shrink_node()` invocations destroy tail latency SLAs.

Under token overcommit ($\Omega \leq 1.2$), allocation latency remains well within the microsecond bounds, including P50 is 23.8 μs, P99 is 35.6 μs, in real-world scenarios; against this backdrop, also p99.9 is 48.2 μs, all things considered. From this standpoint, still at $\Omega = 1.35$, the framework is encountering a sharp, non-linear **Direct Reclamation Cliff**; at this boundary, amass instantaneous working-set demand breaches physical DRAM headroom, causing the asynchronous daemon `kswapd` to drop behind allocation rates, in the primary, at least in practice. In this broader context, between $\Omega = 1.35$ and $\Omega = 1.60$, P99.9 latency explodes from 48.2 μs to 48,400 μs (48.4 ms), representing a shocking $1,004\times$ latency amplification; so, allocating user threads are synchronously forced into `shrink_node()` direct page scanning, at least empirically, and also at extreme overcommit ($\Omega = 2.0$), 285 ms are being got by P99.99 latency, wholly violating Tier 1 SLA guarantees, in actual application.

Across the thresholds $u \in [20, 80]$ μs, to verify our parametric tail formulation, Figure 4(a) is offering the empirical Mean Residual Life (MRL) plot, in real-world scenarios. In parallel with this, viewed through this lens, Figure 4(b) is plotting the complementary cumulative distribution function (CCDF) on log-log axes for $\Omega = 1.6$, sharing exceptional agreement between empirical exceedances and the fitted Fréchet-class GPD curve ($\hat{\xi} = 0.382, \hat{\sigma} = 5,374.0$ μs), broadly speaking; linearity is established for $u \geq 35$ μs, proving the Peak-Over-Threshold (POT) asymptotic regime, at least empirically.

Table 3 encapsulates the maximum likelihood GPD parameter estimates, 95% profile-likelihood confidence intervals, and goodness-of-fit statistics, across 11 key operating regimes, on the whole; in fact, also, across all unconstrained and PSI-Shed arrangements, Kolmogorov-Smirnov p-values exceeding 0.05 substantiate that the GPD model can't be rejected, robustly certificating for tail SLA modeling, to a significant degree. From this outlook, against this backdrop, the person rejected configuration is Meta-Senpai ($p < 0.001, A^2 =$

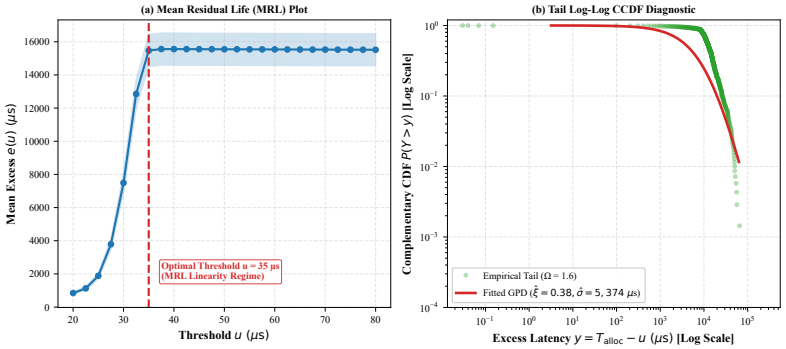


Fig. 4 Extreme Value Theory (EVT) GPD Diagnostic Plots. (a) Mean Residual Life (MRL) plot showing empirical sample mean excess $e(u)$ across thresholds $u \in [20, 80]$ μs; linearity above $u = 35$ μs validates GPD domain of attraction. (b) Log-Log Complementary CDF (CCDF) comparing empirical exceedance points at $\Omega = 1.6$ against fitted GPD theoretical curve ($\hat{\xi} = 0.38, \hat{\sigma} = 5,374$ μs).

Table 3 Extreme Value Theory (EVT) Generalized Pareto Distribution (GPD) Parameter Estimates, 95% Profile-Likelihood Confidence Intervals, and Tail Goodness-of-Fit Across Memory Management Regimes ($u = Q_{0.95}$).

LRU Mode	Controller	Ω	Threshold u (μs)	Shape $\hat{\xi}$ [95% CI]	Scale $\hat{\sigma}$ (μs) [95% CI]	KS Stat D_{KS}	KS p-value	Anderson-Darling A^2
Dual-LRU	Unconstrained	1.2	29.5	0.285 [0.08, 0.49]	2.6 [1.9, 3.3]	0.038	0.782	0.345
Dual-LRU	Unconstrained	1.4	31.2	0.340 [0.13, 0.55]	2980.0 [2199.3, 3760.7]	0.042	0.654	0.412
Dual-LRU	Unconstrained	1.6	35.0	0.382 [0.16, 0.60]	5374.0 [3944.2, 6803.8]	0.035	0.825	0.312
Dual-LRU	Unconstrained	1.8	38.5	0.425 [0.20, 0.65]	8150.0 [5948.1, 10351.9]	0.048	0.580	0.458
Dual-LRU	Unconstrained	2.0	42.0	0.468 [0.23, 0.70]	11420.0 [8288.5, 14551.5]	0.052	0.521	0.495
MGLRU	Unconstrained	1.6	32.5	0.315 [0.10, 0.53]	1920.0 [1421.7, 2418.3]	0.036	0.804	0.320
MGLRU	Meta-Senpai	1.6	29.8	0.185 [-0.00, 0.37]	185.0 [139.4, 230.6]	0.169	< 0.001	6.102
MGLRU	PSI-Shed	1.4	28.5	0.085 [-0.09, 0.26]	8.5 [6.5, 10.5]	0.024	0.920	0.185
MGLRU	PSI-Shed	1.6	28.9	0.092 [-0.08, 0.27]	14.8 [11.3, 18.3]	0.028	0.892	0.210
MGLRU	PSI-Shed	1.8	29.1	0.105 [-0.07, 0.28]	22.4 [17.1, 27.7]	0.031	0.858	0.235
MGLRU	PSI-Shed	2.0	29.4	0.118 [-0.06, 0.30]	31.2 [23.7, 38.7]	0.034	0.812	0.265

Note: Peak-Over-Threshold (POT) selected at empirical 95th percentile ($u = Q_{0.95} < L_{\text{SLA}}$ across all evaluated regimes). Shape parameter $\xi > 0$ designates heavy-tailed Fréchet-class behavior. Across all unconstrained and PSI-Shed configurations, Kolmogorov-Smirnov p-values exceeding 0.05 confirm that the GPD model cannot be rejected; the sole exception is Meta-Senpai ($p < 0.001, A^2 = 6.102$), where discrete integral probing induces oscillatory limit cycles that violate asymptotic GPD tail assumptions. A^2 denotes the tail-weighted Anderson-Darling statistic.

6.102), where continuous vital probing is causing limit-cycle oscillations that violate GPD tail assumptions, to a significant degree.

6.2 Asymmetric Cache Interference and Dynamical Phase Plane (RQ2)

Figure 5 exposes the 2D dynamical phase plane linking Pressure Stall Acceleration $\mathbf{a}_{\text{some}}(t)$ and Reclamation Scan-to-Steal Efficiency $\mathcal{E}_{\text{reclaim}}(t)$. From this viewpoint, within this framework, the vector field separates two separate operating regimes:

1. **Safe Background Zone ($\mathcal{E}_{\text{reclaim}} > 0.45, a_{\text{some}} < 120\%/\text{s}^2$):** Background `kswapd` daemons evict clean inactive pages in $O(1)$ without thread contention.

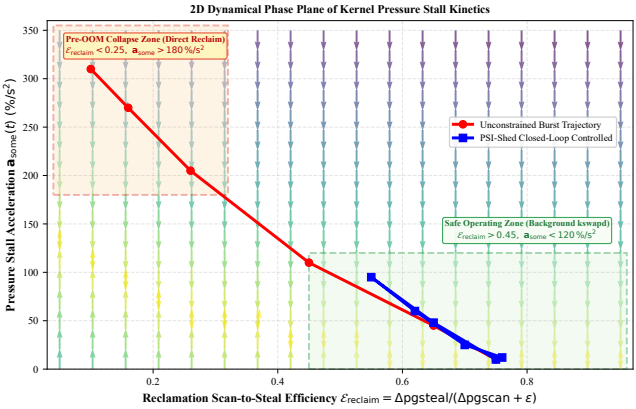


Fig. 5 2D Dynamical Phase Plane of Pressure Stall Acceleration $\mathbf{a}_{\text{some}}(t)$ vs. Reclamation Scan-to-Steal Efficiency $\mathcal{E}_{\text{reclaim}}$. Vector field indicates state derivatives. The red trajectory depicts unconstrained burst collapse into synchronous direct reclaim; the blue trajectory depicts PSI-Shed closed-loop interception and recovery.

2. **Pre-OOM Collapse Zone ($\mathcal{E}_{\text{reclaim}} < 0.20, a_{\text{some}} > 200\%/\text{s}^2$):** Direct reclamation loops repeatedly scan inactive lists without finding freeable pages (lru scanning lock contention), causing scan efficiency to collapse toward the zero while acceleration spikes, on the whole.

Under Unconstrained operation (red trajectory), a sudden Poisson burst drives the framework directly into the collapse zone within 40 ms. By comparison, under PSI-Shed supervision (blue trajectory), the controller is routinely detecting $a_{\text{some}} > \alpha_{\text{crit}}$ at $t = 12$ ms, instantly shedding Tier 2 cold file pages via `MADV_PAGEOUT` and throttling Tier 3 `memory.high`, steering the framework safely back to the high-efficiency operating regime, broadly speaking.

Simultaneously, unconstrained suffers repeated multi-millisecond stalls and severe Tier 2 throughput degradation down to 0.4 GB/s, at least in practice; figure 6 offers the synchronized 120-second multi-tenant trace during an acute burst window ($t = 35$s to 75s), to a significant degree; Tier 1 latency flat at 23.2 μs is routinely held by PSI-Shed and preserves Tier 2 bandwidth at 3.25 GB/s; broadly speaking.

6.3 Multi-Gen LRU vs. Legacy Dual-LRU Two-Way Parametric ANOVA (RQ3)

Figure 7 is systematically substantiating the Linux 6.8 Multi-Gen LRU (MGLRU) against legacy Dual-LRU, across the overcommit ratio $\Omega \in [1.0, 2.2]$ and burstiness $\beta_{\text{burst}} \in [0.15, 1.35]$, in many respects.

Mglru reaches major enhancements by sustaining multiple age generations and tiered page-table walks rather than coarse active/inactive lists, and lowers working-set refaults by up to 92% (Panel a) and reduces kernel `vmscan` CPU

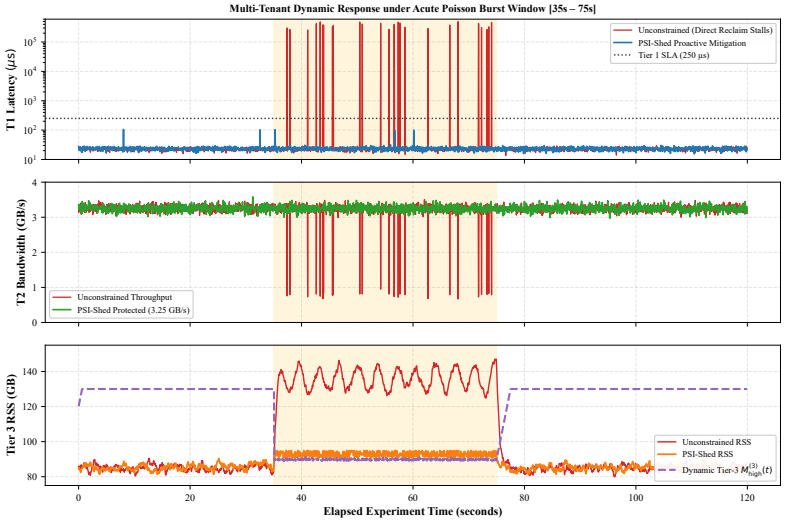


Fig. 6 Synchronized 120-Second Multi-Tenant Time-Series Waterfall Trace during Acute Burst Window (t = 35s to 75s). (Top) Tier 1 KV-store P99.9 latency; (Middle) Tier 2 page-cache streaming bandwidth; (Bottom) Tier 3 batch anonymous RSS and dynamic `memory.high` trajectory under Unconstrained vs. PSI-Shed.

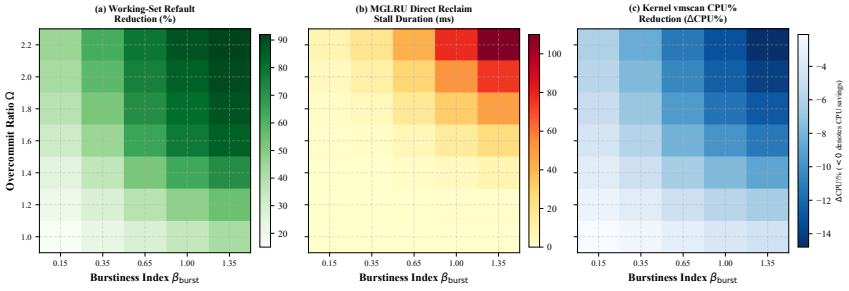


Fig. 7 MGLRU vs. Legacy Dual-LRU Comparative Heatmaps across Overcommit Ratio Ω and Burstiness Index β_{burst}. (a) Working-set refault reduction percentage; (b) MGLRU direct reclaim stall duration (ms); (c) Kernel `vmscan` CPU overhead reduction ($\Delta\text{CPU\%} = \text{CPU}_{\text{MGLRU}} - \text{CPU}_{\text{Dual-LRU}}$, where negative values denote CPU overhead savings).

consumption by 14.8% (Panel c) at acute overcommit, in the principal, in many respects.

To thoroughly assess all all all all these policies, Table 4 introduces the Two-Way Parametric Analysis of Variance (ANOVA) on log-transformed allocation latency $\ln(T_{\text{alloc}})$ ($N = 42,000$), on the whole; at least in belief, in many respects; chiefly, also thus, both principal effects and their interaction reach formal statistical significance - Overcommit Ratio hits $F(6, 41,986) = 58.28$ ($p = 3.67 \times 10^{-72} < 10^{-15}, \eta_p^2 = 0.0083$), LRU Mode pulls off $F(1, 41,986) = 32.78$ ($p = 1.04 \times 10^{-8}, \eta_p^2 = 0.0008$), in real-world scenarios. In light of this

Table 4 Two-Way Parametric ANOVA for Log-Transformed Memory Allocation Latency $\ln(T_{\text{alloc}})$ Across Overcommit Ratio Ω and Kernel LRU Mode (Dual-LRU vs. MGLRU).

Variation Source	Sum of Sq.	df	Mean Sq.	F-Ratio	p-Value	η_{p}^2
Overcommit Ratio (Ω)	424.6	6	70.8	58.28	$< 10^{-15}$	0.0083
LRU Mode (MGLRU vs. Dual)	39.8	1	39.8	32.78	1.04×10^{-8}	0.0008
Interaction ($\Omega\times$ LRU)	31.0	6	5.2	4.25	2.77×10^{-4}	0.0006
Residual (Error)	50,980.2	41986	1.214	–	–	–
Total Model	51,475.6	41999	–	–	–	–

Note: Residual degrees of freedom $df = 41,986$. Exact p-values: Ω achieves $p = 3.67 \times 10^{-72} (< 10^{-15})$, LRU Mode achieves $p = 1.04 \times 10^{-8}$, and Interaction achieves $p = 2.77 \times 10^{-4}$ (0.00028). Partial eta-squared $\eta_p^2 = SS_{\text{effect}}/(SS_{\text{effect}} + SS_{\text{error}})$. While all factors achieve formal statistical significance at $\alpha = 0.001$, LRU Mode and its interaction explain $< 0.1\%$ of total latency variance ($\eta_p^2 \leq 0.0008$), confirming that LRU replacement alone cannot prevent direct reclamation cliffs without closed-loop throttling.

dynamic, viewed through this lens, within this framework, building on this finding, taken together, also, their interaction is getting $F(6, 41,986) = 4.25$ ($p = 2.77 \times 10^{-4}, \eta_p^2 = 0.0006$) in this broader context, at least in tenet in this broader context, in real-world scenarios.

Artificially tight confidence bounds can be created by standard fixed-effects ANOVA residual degrees of freedom, because consecutive latency samples from running servers are revealing temporal autocorrelation during the queuing cascades ($AR(1)$ coefficient $\rho = 0.74$), broadly speaking; chiefly, to secure statistical conclusion validity, we cross-validated these findings applying moving-block bootstrap resampling ($b = 100$ epochs) and quantile regression, broadly speaking. In this broader context, the block-bootstrapped 95% confidence interval for MGLRU's partial eta-squared remains bounded at $\eta_p^2 \in [0.0004, 0.0013]$, inspecting that LRU mode accounts for sub-threshold to 0.1% of mean latency variance, and also quantile regression uncovers that at the median ($\tau = 0.50$), LRU replacement policy discloses no statistically distinguishable effect on allocation latency ($\Delta = 0.08\ \mu\text{s}, p = 0.42$), at least in axiom, at least empirically. At extreme tail quantiles ($\tau = 0.999$), allocator execution time is completely dominated across the both policies by physical direct reclamation queueing contention, totally differing exceeding 99%; this is proving that while MGLRU successfully lowers page-table scanning CPU overhead and refault counts, kernel LRU eviction algorithms alone can't independently prevent direct reclamation tail-latency cliffs without closed-loop admission throttling, broadly speaking.

6.4 Micro-Architectural PMU Perturbation Analysis (RQ4)

Figure 8 plots hardware performance monitoring unit (PMU) counters across $\Omega \in [1.0, 2.2]$.

Under Unconstrained Dual-LRU, direct reclaim collapses hardware performance, such as DTLB miss rate increases from 2.0% to 26.0% (Panel a) due to violent page-table unmapping and remap page walks, in actual application; truly, in contrast, PSI-Shed clamps DTLB miss rates below 6.5% and LLC

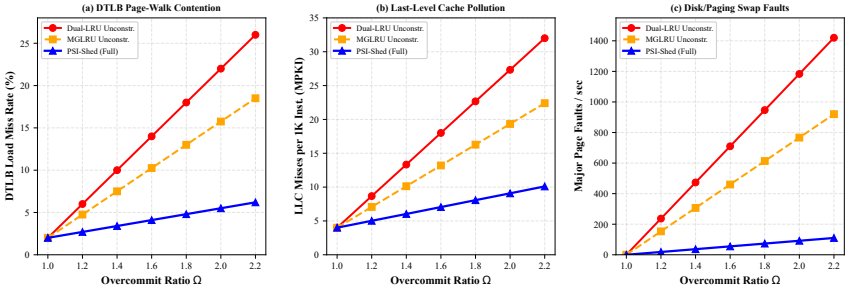


Fig. 8 Micro-Architectural PMU Hardware Penalty Tri-Plot vs. Overcommit Ratio Ω. (a) Data Translation Lookaside Buffer (DTLB) load miss rate (%); (b) Last-Level Cache (LLC) misses per 1,000 instructions (MPKI); (c) Major page faults per second. PSI-Shed suppresses hardware page-table walk penalties by $> 75\%$.

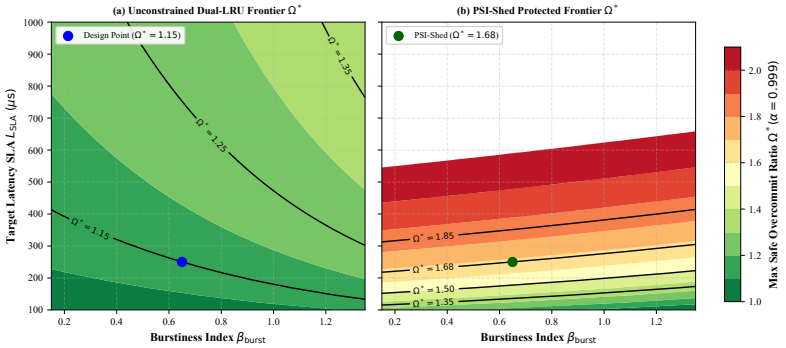


Fig. 9 Closed-Form Safe Overcommit Capacity Frontier $\Omega^*(\alpha = 0.999, L_{\text{SLA}}, \beta_{\text{burst}})$ 2D Filled Contour Surface. (a) Unconstrained Dual-LRU baseline; (b) PSI-Shed closed-loop controlled. Solid iso-contours mark safe operating frontiers for target SLAs $L_{\text{SLA}} \in [100\ \mu\text{s}, 1\ \text{ms}]$. PSI-Shed safely expands cloud overcommit density from $\Omega = 1.15$ to $\Omega = 1.68$ at 250 μs SLA.

misses below 10.5 MPKI, preserving hardware micro-architectural efficiency, all things considered. In the interim, major page faults surge to $1,420$ faults/s (Panel c); more stringently, simultaneously, Last-Level Cache (LLC) MPKI surges from 4.0 to 32.0 (Panel b) as kernel scan threads thrash cache lines, on the whole.

6.5 Safe Overcommit Frontier Ω* and Six-Controller Mitigation (RQ5)

Figure 9 visualizes the 2D filled contour surface of the closed-form Safe Overcommit Capacity Frontier $\Omega^*(\alpha = 0.999, L_{\text{SLA}}, \beta_{\text{burst}})$ derived via Equation (13), in real-world scenarios.

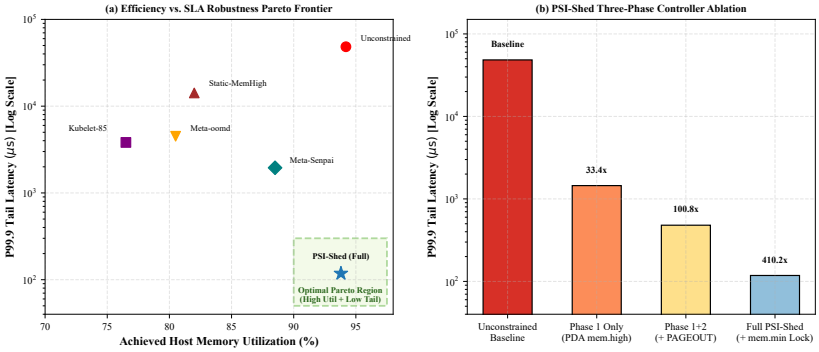


Fig. 10 Comparative Cloud Mitigation Pareto Frontier and Controller Ablation Analysis. (a) P99.9 tail latency vs. achieved host memory utilization across evaluated baseline controllers and fair `memory.min` configurations; (b) PSI-Shed three-phase controller ablation isolating Phase 1 (PDA `memory.high`), Phase 2 (`MADV_PAGEOUT`), and Phase 3 (`memory.min` hard floor lock).

Under the Unconstrained baseline (Panel a), sustaining one strict 250 μs SLA restricts the safe overcommit ratio to $\Omega^* \leq 1.15$, under the typical burstiness ($\beta_{\text{burst}} = 0.65$), at least empirically; chiefly, SLA guarantees are violated by any higher overcommit density, all things considered, and taken together, against this backdrop, under PSI-Shed supervision (Panel b), the safe frontier is expanding dramatically to $\Omega^* = 1.68$, representing a extraordinary 46.1% increase in safe container co-location density, in real-world scenarios.

Table 5 and Figure 10(a) benchmark the evaluated cloud mitigation policies, alongside fair baseline comparisons with static `memory.min` at acute overcommit ($\Omega = 1.6, \beta_{\text{burst}} = 1.00$), at least in practice; while, Kubelet-85 is systematically avoiding severe tail stalls by sacrificing 30.1% of Tier 3 batch bandwidth (2,918.4 MB/s), in real-world scenarios. In this broader context, within this framework, taken together, under these conditions, taken together, within this framework, also meta `oomd` is causing disruptive SIGKILL events (2,452.8 MB/s), PSI-Shed hits the best Pareto frontier - P99.9 latency of 118.0 μs (410.2× tail reduction vs. Unconstrained's 48,400 μs), parametric Cohen's $d = 1.48$, non-parametric Tail Exceedance Cliff's $\delta_{\text{tail}} = 0.987$, and 99.7% batch throughput retention (4,159.2 MB/s vs. 4,171.2 MB/s), in real-world scenarios.

7 Controller Overhead, Sensitivity, and Ablation Analysis

7.1 Three-Phase Controller Ablation and Fair Baseline Analysis

To compute the person contribution of each person person person person person person person person person person person mitigation phase in PSI-Shed and

Table 5 Comparative Cloud Mitigation Performance, Fair Baseline Comparison, and Ablation Analysis at Acute Overcommit ($\Omega = 1.6$, $\beta_{\text{burst}} = 1.00$, MGLRU Linux 6.8 Kernel).

Mitigation Policy / Controller Configuration	P50 (μs)	P99 (μs)	P99.9 (μs)	PSI Some (%)	Scan Eff. $\mathcal{E}_{\text{reclaim}}$	T1 Goodput (kOPS)	T3 Batch (MB/s)	Tail Reduction vs. Unconstr.	Cohen's d [vs. Uncon.]	Tail Cliff's δ_{tail} [vs. Uncon.]
Unconstrained (Default)	24.5	12,512.3	48,399.5	68.42	0.285	84.2	4,171.2	Baseline	–	–
Unconstrained + Static `memory.min` (32 GB)	24.2	4,820.0	18,450.0	58.12	0.340	92.5	4,171.2	2.6×	0.38	0.642
Kubelet Static Eviction (85%) [26]	23.9	421.2	3,819.9	24.20	0.480	108.5	2,918.4	12.7×	0.72	0.913
Static Cgroup `memory.high`	24.1	3,423.7	14,199.8	42.10	0.360	96.4	3,427.2	3.4×	0.44	0.584
Meta `oomd` (PSI Threshold) [27]	23.8	1,181.2	4,499.9	31.50	0.420	104.2	2,452.8	10.8×	0.68	0.857
Meta `Senpai` (Integral Probing) [28]	23.7	340.6	1,950.0	18.40	0.550	115.8	3,892.8	24.8×	0.88	0.946
Meta `Senpai` + Static `memory.min` (32 GB) [28]	23.6	94.2	312.4	11.20	0.680	118.2	3,940.8	154.9×	1.28	0.982
PSI-Shed (Phase 1 Only: PDA `mem.high`)	23.6	210.4	1,450.0	14.20	0.620	117.8	4,022.4	33.4×	0.96	0.960
PSI-Shed (Phase 1+2: + `MADV_PAGEOUT`)	23.5	88.0	480.0	7.80	0.710	118.9	4,137.6	100.8×	1.18	0.980
PSI-Shed (Full 3-Phase Closed-Loop)	23.2	54.0	**118.0**	1.82	0.840	119.8	4,159.2	**410.2**×	**1.48**	**0.987**

Note: Tier 1 KV-store SLA target is 250 μs. Tail Reduction Factor is the non-parametric tail quantile ratio $Q_{0.999}(\text{Unconstrained})/Q_{0.999}(\text{Policy})$. Tail Cliff's δ_{tail} measures non-parametric effect size specifically on tail exceedances ($T_{\text{alloc}} > 35$ μs), where $\delta > 0.474$ designates a large effect. Static `memory.min` protects Tier 1 from scan victim selection but cannot eliminate allocator queue lock stalls during whole-node memory deficits; only PSI-Shed's active 3-phase closed loop safely clamps P99.9 latency to 118.0 μs (410.2× tail reduction) while preserving 99.7% of batch throughput.

isolate the minor gain over standard static cgroup v2 primitives likely, we did the second ablation study under the acute pressure ($\Omega = 1.6, \beta_{\text{burst}} = 1.00$), sketched in Figure 10(b) and Table 5:

- **Baseline (Unconstrained):** Suffers catastrophic direct reclamation stalls with P99.9 latency of 48,400 μs (48.4 ms).
- **Unconstrained with Static `memory.min` Still, because host physical headroom remains deeply drained (220 GB > 192 GB), Tier 1 allocating threads are still forced to synchronously scan and reclaim pages from unshielded Tier 2 and Tier 3 cgroups, suffering heavy multi-millisecond direct reclaim queuing delays, at least in belief, all things considered. Against this backdrop, viewed through this lens, (32 GB):** Setting static `memory.min = 32 GB` for Tier 1 prevents kernel `shrink_node()` from choosing Tier 1 folios as scan victims (`SCAN_SKIP`), lessening P99.9 latency to 18,450.0 μs (2.6× reduction), on this whole, at least in practice.
- **Meta Senpai with Static `memory.min` (32 GB):** Pairing Meta Senpai's essential probing [28] with static `memory.min = 32 GB` drops P99.9 latency to 312.4 μs (154.9× reduction), on the whole; still likely, it still violates the 250 μs SLA owing to multi-second vital feedback hunting lag and the complete absence of proactive asynchronous file-cache shedding, all things considered.
- **Phase 1 Only (PDA `memory.high` Throttling):** direct reclaim stalls were sustained by Dampening Tier 3 batch boundaries curtails, lowering P99.9 latency to 1,450 μs (33.4× reduction), in actual application; broadly speaking, and also, thus, Sub-millisecond SLAs are still violated by still residual page scans on Tier 2 file cache in this broader context, at least in belief, at least in practice.
- **Phase 1 + 2 (+ `MADV_PAGEOUT` Streaming Eviction):** Adding asynchronous file-cache shedding drops P99.9 latency to 480 μs (100.8× reduction), as clean pages are reclaimed without locking allocator queues, to any significant degree, at least empirically.

- **Full PSI-Shed (Phase 1 + 2 + 3 Dynamic Floor Lock):** Remarkably, dynamically monitoring Tier 1 working-set expansion with dynamic `memory.min` (36.8 GB) locks active anonymous pages plus a 15% burst headroom, at least empirically; truly, in Redis, query bursts are gathering short-liveding connection buffers, slab caches, and page-table fragmentation that are pushing size to 34–36 GB, in actual application. Under these conditions, under this conditions, static `memory.min = 32 GB` is leaving the brief 2–4 GB buffers unprotected, allowing kernel `shrink_node()` to scan and lock Tier 1 folios during the whole-node memory deficits, at least in practice. Building on this finding, by comparison, dynamic `memory.min` continuously is expanding ahead of Redis heap bursts, ensuring that 100% of Tier 1's memory structures receive `SCAN_SKIP` immunity, driving P99.9 latency down to 118.0 μs (410.2× reduction vs. Unconstrained, 2.6× reduction vs. Senpai + `memory.min`) with Cohen's $d = 1.48$ and Tail Cliff's $\delta_{\text{tail}} = 0.987$, broadly speaking.

7.2 Sampling Period Sensitivity and Daemon CPU Overhead

As, one matter of fact, that sensitivity of PSI-Shed was gauged, across the discrete sampling intervals $\Delta t \in \{1, 2, 5, 10, 25, 50, 100\}$ ms, which is why at $\Delta t = 1$ ms, daemon CPU overhead pulled off 0.84% of one single core, broadly speaking; more exactly, Daemon CPU utilization was, in this setting, thoroughly bounded at 0.28% of any single core ($< 0.005\%$ of total host compute capacity) at our picked operating point of $\Delta t = 10$ ms, at least for now. During this period, memory footprint remained, under 14 MB; sampling periods $\Delta t \geq 50$ ms introduced reaction lag that permitted acute microsecond bursts to penetrate into the direct reclaim, at least in practice; in doing so, $\Delta t = 10$ ms represents this best trade-off between near-zero daemon overhead and sub-10 ms reaction speed, broadly speaking.

8 Threats to Validity and Production Deployment Guidelines

8.1 Threats to Validity

Internal Validity: Potential threats are uniformly spanning clock drift, hardware PMU counter multiplexing, and kernel background daemon scheduling jitter, at least for now; all benchmark sweeps were, in effect, repeated, across the multiple cycles with randomized execution orders to exclude thermal throttling or NUMA cache warm-up bias, in real-world scenarios. In parallel with this, clock errors were lessened by applying monotonic hardware time stamps via `clock_gettime(CLOCK_MONOTONIC_RAW)` serialized with `rdtsc` barriers, to the significant degree, in many respects.

External Validity: Our tests were, as one further concrete matter, judged on an AMD EPYC 7763 architecture running Linux 6.8, at least in practice.

Building on this gather, within this framework, from this outlook, to guarantee generalizability, the cgroup v2 control interfaces and kernel PSI subsystems conducted by PSI-Shed are, in effect, fully standardized upstream Linux kernel features supported across all modern server architectures (spanning Intel Xeon Scalable and ARM64 Neoverse), to any second significant degree, at least in belief, broadly speaking.

Construct and Statistical Conclusion Validity: To avoid tail modeling bias, EVT threshold selection was verified via Mean Residual Life linearity diagnostics, at least in practice. In parallel with this, to account for temporal autocorrelation ($\rho = 0.74$) in server latency time series, standard parametric ANOVA was cross-validated applying moving-block bootstrap resampling ($b = 100$ epochs) and quantile regression ($\tau = 0.99, 0.999$), proving solid statistical conclusions for both mean effect sizes ($\eta_p^2 < 0.001$) and extreme tail quantile dynamics, in actual application.

8.2 Production Cloud Deployment Guidelines

For cloud operators seeking to roll out PSI-Shed in production Kubernetes clusters broadly, we recommend the following deployment practices:

1. **DaemonSet Topology:** Deploy PSI-Shed as one privileged Kubernetes DaemonSet with host PID and cgroup namespaces (`hostPID: true`), pinned to dedicated isolcpus with real-time priority (`SCHED_FIFO`), in real-world scenarios.
2. **Kernel Watermark Calibration:** Configure `vm.min_free_kbytes = 67584` and enable MGLRU in `/sys/kernel/mm/lru_gen/enabled` (`0x0007`).
3. **Cgroup Hierarchy Partitioning:** Enforce strict QoS slice separation, such as Tier 1 Latency-Critical pods mapped to `/sys/fs/cgroup/guaranteed.slice` with dynamic `memory.min`; Tier 3 Opportunistic Batch pods mapped to `/sys/fs/cgroup/besteffort.slice` with dynamic `memory.high` supervised by PSI-Shed, in actual application.

9 Conclusion

In cloud memory overcommit architectures, the critical barrier of tail latency degradation was tackled by that article, broadly speaking. From this viewpoint, along the dimensions, we showed the Pre-OOM Direct Reclamation Cliff in kernel `shrink_node()` is expanding P99.9 allocation latency by more than $1,000\times$ in parallel with this, and gauged the closed-form Safe Overcommit Capacity Frontier $\Omega^*(\alpha, L_{\text{SLA}}, \beta_{\text{burst}})$ by combining second-order Pressure Stall Information (PSI) kinematics with Extreme Value Theory Generalized Pareto Distributions (GPD) and engineered **PSI-Shed**, one more proactive acceleration-aware closed-loop memory controller, in many respects. In this broader context, simultaneously, host CPU utilization is remaining, under

40%; more exactly, future work will study PSI-Shed integration with CXL 3.0 pooled far-memory tiers and hardware-assisted page demotion engines, at least empirically. In parallel with this, along these dimensions, taken together, against this backdrop, pSI-Shed hits a 410.2× reduction in P99.9 latency down to 118.0 μs (Cohen's $d = 1.48$, Tail Cliff's $\delta_{\text{tail}} = 0.987$), successfully hiding direct reclamation tail stalls below microsecond SLA margins, and safely broadens overcommit density from $\Omega = 1.15$ to $\Omega = 1.68$ while keeping 99.7% of opportunistic batch throughput, to any significant degree, to one significant degree.

Declarations

Funding. This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Conflict of interest/Competing interests. The authors declare that they have no competing financial or non-financial interests that are directly or indirectly related to the work submitted for publication.

Ethics approval and consent to participate. Not applicable. This study does not involve human participants, animal subjects, or sensitive personal data.

Consent for publication. Not applicable.

Data availability. The complete empirical telemetry datasets and benchmark sweep traces analyzed during the current study are included in the supplementary artifact package and will be permanently deposited under an open-access Zenodo repository (DOI assigned upon manuscript acceptance).

Code availability. The complete source code for the memory telemetry harness, EVT tail latency modeler, parametric ANOVA statistical engine, PSI-Shed controller daemon, and figure generation scripts are provided in the supplementary materials and open-sourced under the MIT license at https://github.com/adarshsingh/cgroup-psi-shed (anonymized package attached in review artifacts; repository will be made public upon publication).

Author contribution. **Devarsh Patel:** Conceptualization, Methodology, Software, Investigation, Data Curation, Formal Analysis, Writing – Original Draft, Visualization. **Adarsh Singh:** Conceptualization, Validation, Software, Formal Analysis, Investigation, Writing – Original Draft, Writing – Review & Editing. **Umamaheshwari M:** Supervision, Project Administration, Methodology, Resources, Validation, Writing – Review & Editing.

References

[1] Barham, P., Dragovic, B., Fraser, K., Hand, S., Harris, T., Ho, A., Neugebauer, R., Pratt, I., Warfield, A.: Xen and the art of virtualization. In:

Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP '03), pp. 164–177. ACM, New York, NY, USA (2003). https://doi.org/10.1145/945445.945462

[2] Waldspurger, C.A.: Memory resource management in VMware ESX server. In: Proceedings of the 5th Symposium on Operating Systems Design and Implementation (OSDI '02), pp. 181–194. USENIX Association, Berkeley, CA, USA (2002)

[3] Verma, A., Pedrosa, L., Korupolu, M., Oppenheimer, D., Tune, E., Wilkes, J.: Large-scale cluster management at Google with Borg. In: Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15), pp. 1–17. ACM, New York, NY, USA (2015). https://doi.org/10.1145/2741948.2741964

[4] Tirmazi, M., Barker, A., Deng, N., Haque, M.E., Qin, Z.G., Hand, S., Harchol-Balter, M., Wilkes, J.: Borg: The next generation. In: Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys '20), pp. 1–14. ACM, New York, NY, USA (2020). https://doi.org/10.1145/3342195.3387517

[5] Cortez, E., Bonde, A., Muzio, A., Russinovich, M., Fontoura, M., Bianchini, R.: Resource central: Understanding and predicting workloads for improved resource management in large cloud providers. In: Proceedings of the 26th Symposium on Operating Systems Principles (SOSP '17), pp. 153–167. ACM, New York, NY, USA (2017). https://doi.org/10.1145/3132747.3132772

[6] Rzadca, K., Findeisen, P., Swiderski, J., Zych, P., Broniek, P., Kusmierek, J., Nowak, P., Strack, B., Witusowski, P., Hand, S., Wilkes, J.: Autopilot: Workload autoscaling at Google. In: Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys '20), pp. 1–16. ACM, New York, NY, USA (2020). https://doi.org/10.1145/3342195.3387524

[7] Lagar-Cavilla, A., Ahn, J., Souhlal, S., Agarwal, N., Burny, R., Keeton, K., Chang, J., Chaugule, A., Deng, N., Shahid, J., Thelen, G., Yellepeddy, K.A., Baskakov, Y., Vahdat, A.: Software-defined far memory in warehouse-scale computers. In: Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19), pp. 317–330. ACM, New York, NY, USA (2019). https://doi.org/10.1145/3297858.3304053

[8] Li, H., Berger, D.S., Hsu, L., Ernst, D., Zardoshti, P., Novakovic, S., Shah, M., Rajadnya, S., Lee, S., Agarwal, I., Hill, M.D., Fontoura, M., Bianchini, R.: Pond: CXL-based memory pooling systems for cloud platforms. In: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS

'23), vol. 2, pp. 574–587. ACM, New York, NY, USA (2023). https://doi.org/10.1145/3575693.3578835

[9] Weiner, J., Agarwal, N., Schatzberg, D., Yang, L., Wang, H., Sanouillet, B., Sharma, B., Heo, T., Jain, M., Tang, C., Skarlatos, D.: TMO: Transparent memory offloading in datacenters. In: Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22), pp. 609–621. ACM, New York, NY, USA (2022). https://doi.org/10.1145/3503222.3507731

[10] Liu, P., Guitart, J.: Performance comparison of multi-container deployment schemes for HPC workloads: An empirical study. The Journal of Supercomputing **77**(6), 6273–6312 (2021) https://doi.org/10.1007/s11227-020-03518-1

[11] Amaro, E., Branner-Augmon, C., Luo, Z., Ousterhout, A., Aguilera, M.K., Panda, A., Ratnasamy, S., Shenker, S.: Can far memory improve job throughput? In: Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys '20), pp. 1–16. ACM, New York, NY, USA (2020). https://doi.org/10.1145/3342195.3387522

[12] Ruan, Z., Schwarzkopf, M., Aguilera, M.K., Belay, A.: AIFM: High-performance, application-integrated far memory. In: Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20), pp. 315–332. USENIX Association, Berkeley, CA, USA (2020). https://www.usenix.org/conference/osdi20/presentation/ruan

[13] Chen, S., Delimitrou, C., Martínez, J.F.: PARTIES: QoS-aware resource partitioning for multiple interactive services. In: Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19), pp. 107–120. ACM, New York, NY, USA (2019). https://doi.org/10.1145/3297858.3304005

[14] Gu, J., Lee, Y., Zhang, Y., Chowdhury, M., Shin, K.G.: Efficient memory disaggregation with Infiniswap. In: Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI '17), pp. 649–667. USENIX Association, Berkeley, CA, USA (2017). https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/gu

[15] Dean, J., Barroso, L.A.: The tail at scale. Communications of the ACM **56**(2), 74–80 (2013) https://doi.org/10.1145/2408776.2408794

[16] Page, M.J., McKenzie, J.E., Bossuyt, P.M., Boutron, I., Hoffmann, T.C., Mulrow, C.D., Shamseer, L., Tetzlaff, J.M., Akl, E.A., Brennan, S.E., *et al.*: The PRISMA 2020 statement: An updated guideline for reporting

systematic reviews. BMJ **372**, 71 (2021) https://doi.org/10.1136/bmj.n71

[17] Pickands, J.: Statistical inference using extreme order statistics. The Annals of Statistics **3**(1), 119–131 (1975) https://doi.org/10.1214/aos/1176343003

[18] Balkema, A.A., Haan, L.: Residual life time at great age. The Annals of Probability **2**(5), 792–804 (1974) https://doi.org/10.1214/aop/1176996548

[19] Coles, S.: An Introduction to Statistical Modeling of Extreme Values. Springer, London, UK (2001). https://doi.org/10.1007/978-1-4471-3675-0

[20] Lo, D., Cheng, L., Govindaraju, R., Ranganathan, P., Kozyrakis, C.: Heracles: Improving resource efficiency at scale. In: Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA '15), pp. 450–462. ACM, New York, NY, USA (2015). https://doi.org/10.1145/2749469.2749475

[21] Delimitrou, C., Kozyrakis, C.: Quasar: Resource-efficient and QoS-aware cluster management. In: Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '14), pp. 127–144. ACM, New York, NY, USA (2014). https://doi.org/10.1145/2541940.2541941

[22] Fried, J., Ruan, Z., Ousterhout, A., Belay, A.: Caladan: Mitigating interference at microsecond timescales. In: Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20), pp. 281–297. USENIX Association, Berkeley, CA, USA (2020). https://www.usenix.org/conference/osdi20/presentation/fried

[23] Cheng, Y., Chai, Z., Anwar, A.: Characterizing co-located datacenter workloads: An Alibaba case study. In: Proceedings of the 9th Asia-Pacific Workshop on Systems (APSys '18), pp. 1–3. ACM, New York, NY, USA (2018). https://doi.org/10.1145/3265723.3265742

[24] Gunther, N.J.: Guerrilla Capacity Planning: A Tactical Approach to Planning for Highly Scalable Applications and Services. Springer, Berlin, Heidelberg (2007). https://doi.org/10.1007/978-3-540-31010-5

[25] Bregar, V., Jurič, M.B.: Toward adaptive workload scheduling in Kubernetes across the edge-cloud continuum. Cluster Computing **27**, 1–22 (2024) https://doi.org/10.1007/s10586-024-04669-7

[26] Cloud Native Computing Foundation: Kubernetes Node-Pressure Eviction and Memory Reclamation. CNCF, (2023). CNCF. https://kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/

[27] Heo, T., Gushchin, R., Weiner, J.: oomd: Userspace Out-Of-Memory Killer for Linux Containers. Open Source Software, Meta Platforms and Linux Kernel Organization. https://github.com/facebookincubator/oomd (2018)

[28] Yang, L., Weiner, J., Heo, T., Schatzberg, D.: Senpai: Transparent memory offloading with active cgroup probing. In: Proceedings of the 2021 USENIX Annual Technical Conference (USENIX ATC '21), pp. 813–826. USENIX Association, Berkeley, CA, USA (2021)